

RESEARCH ARTICLE

Efficient Task Scheduling and Load Balancing in Fog Computing for Crucial Healthcare Through Deep Reinforcement Learning

PRASHANTH CHOPPARA^{ID} AND BOMMAREDDY LOKESH^{ID}, (Member, IEEE)

School of Computer Science and Engineering, VIT-AP University, Amaravati, Andhra Pradesh 522237, India

Corresponding author: Bommareddy Lokesh (lokesh.bommareddy@vitap.ac.in)

ABSTRACT In healthcare, real-time decision making is crucial for ensuring timely and accurate patient care. However, traditional computing infrastructures, with their wide ranging capabilities, suffer from inherent latency, which compromises the efficiency of time-sensitive medical applications. This paper explores the potential of fog computing to better address this challenge, proposing a new framework that uses deep reinforcement learning (DRL) to advance task scheduling in crucial healthcare. The paper addresses the limitations of cloud computing systems. It proposes and replaces a fog computing architecture in supporting low latency for healthcare applications. This architecture reduces transmission latency by placing processing nodes close to the source of data generation, namely IoT-enabled healthcare devices. The foundation of this approach is the DRL model, which is designed to dynamically optimize the partition of computational tasks across fog nodes to improve both data throughput and operational response times. The effectiveness of the proposed DRL based fog computing model is validated with a series of simulations performed with the SimPy simulation environment. In such simulations, diverse healthcare scenarios, ranging from continuous patient monitoring systems to crucial emergency response applications, are recreated, providing a rich framework for testing the real-time processing capabilities of the model. This algorithm, DRL, has been fine-tuned and extensively implemented in these scenarios to show how the algorithm controls and optimizes tasks and their urgency in accordance with resource demand. By dynamically learning from real-time system states and optimizing task allocation to minimize delays, the DRL model reduces the makespan by up to 30% compared to traditional scheduling approaches. Comparative performance analysis indicated a 30% reduction in task completion times, a 40% reduction in operational latency, and a 25% improvement in fault tolerance relative to traditional scheduling approaches. The flexibility of the DRL model is further considered through its application to diverse real-time data processing contexts in industrial automation and smart traffic systems.

INDEX TERMS Fog nodes, task scheduling, DRL, health care, makespan, trust, fault tolerance, energy.

I. INTRODUCTION

Cloud computing has emerged as the leading solution for remote data storage, processing, and management, minimizing the need for extensive on-site network infrastructure. Its main advantages include flexibility in space utilization, cost efficiency, and simple access, so it is well suited for big

data and diverse purposes [1]. Cloud computing, centralized control is widespread because it simplifies administration, management, and updates. However, it falls short in terms of real-time and low-latency response, which is crucial in many applications. Data collection from distributed systems such as Internet-of-Things sensors and surveillance cameras to a central server can lead to delays and increased bandwidth demands, as the volume of data generated [2] continues to grow. Delays in the execution of tasks in healthcare can

The associate editor coordinating the review of this manuscript and approving it for publication was Nitin Gupta^{ID}.

lead to a number of problems. To address these difficulties, fog computing has developed as an example for executing data processing activities at the network edge. [3]. Real-time processing by decentralizing computations at network edge nodes, devices, and gateways. Such a configuration allows for reduced latency in data transfer to distant cloud hubs, reviving healthcare, automotive, and industrial IoT [4]. Fog computing helps process data locally or across multiple geographic regions, thereby reducing the workload on centralized networks and using bandwidth, which is cost-effective, especially in areas where networks are scarce and expensive, such as rural healthcare centers or industries. With many connected devices [5].

Proper task scheduling is essential in fog computing, which directly affects the scope, resource utilization, and interaction in a fog environment. Unlike traditional cloud computing, which processes tasks in data centers, fog computing relies on edge devices and fog nodes to bring computation closer to the data [6]. This approach allows for near-simultaneous data transmission and processing but introduces challenges due to limited resources and uneven allocation. Effective task scheduling improves task performance, reduces latency, and optimizes resource usage [7]. In fog, task scheduling prioritizes resource usage to improve processing power, memory, bandwidth, and energy consumption. Fog nodes have fewer resources than cloud data centers and must handle multiple operations, applications, and flows from connected IoT devices, sensors, and edge devices. Poor scheduling can overload some nodes, creating bottlenecks or crashes, while others remain unused. An efficient scheduling method is crucial to balance loads across nodes, prevent overloads, and maximize available resources [8]. Reinforcement Learning (RL) introduces dynamic decision-making in task scheduling by continuously learning optimal allocation strategies based on resource availability and workload distribution. This adaptability minimizes delays and ensures efficient task execution, thereby reducing the makespan [9]. For example, a healthcare monitoring system can speed up the analysis of vital signs data. In this case, non-crucial tasks are executed on less crucial network nodes as part of a priority-based scheduling approach [10].

In fog computing for healthcare, four key metrics are makespan, trust, fault tolerance, and energy consumption. Deep reinforcement learning (DRL) plays a important role in optimizing them collectively. Makespan, or task completion time, is particularly important in healthcare environments, where reducing delays in processing and decision making can have significant impacts on patient outcomes. DRL helps by dynamically learning optimal task scheduling policies based on real-time system states, which helps reduce overall makespan by allocating resources more efficiently. However, this time reduction must be balanced with energy consumption, as the system must operate efficiently without overusing resources, which increases operational costs and reduces system longevity. DRL can address this trade-off by adjusting task distribution or optimizing node utilization to

save energy without sacrificing performance. At the same time, healthcare applications depend on reliable and secure operation, so maintaining trust is crucial. If the system is inefficient or prone to failures, trust in the system is lost. DRL increases trust by enabling the system to detect and prevent potential failures early, ensuring that decisions are made with a focus on system stability and reliability. It is closely related to fault tolerance, which ensures that the system continues to operate despite node failures or outages. By learning the best strategies for dynamic task migration or resource redeployment in the event of failures, DRL helps ensure that tasks are completed even when part of the system is unavailable. DRL ensures that fault tolerance policies are implemented in the most energy-efficient manner by avoiding unnecessary redundancy or resource stress by learning from historical data and continuously adapting to the environment.

DRL optimizes these four metrics makespan, trust, fault tolerance, and energy consumption by learning to make trade-off decisions that improve the overall system's performance. The algorithm self-adjusts based on feedback from the environment, ensuring that task scheduling, resource allocation, and system reliability are balanced for optimal healthcare service delivery. By integrating these objectives, DRL can significantly enhance the efficiency and sustainability of fog computing systems, particularly in time-sensitive healthcare applications.

The unique contributions of this study are as follows:

- We formulated a Deep Deterministic Policy Gradient (DDPG) algorithm in fog computing to reduce makespan and improve real-time processing capability crucial to healthcare systems.
- Implementing a healthcare-specific task prioritization approach that assesses and prioritizes tasks based on crucial real-time parameters such as heart rate and ECG values, processing crucial tasks quickly.
- The proposed framework improves the utilization of fog node capacities, reduces transmission delay and increases responsiveness for time-sensitive operations in healthcare.
- The DDPG algorithm optimizes metrics including makespan, energy efficiency, trust, and fault tolerance
- The framework for various healthcare scenarios such as patient monitoring and emergency response systems, with potential for other domains involves real-time processing.

The remaining paper is organized into Section II Related Work, Section III Proposed Methodology, Section IV Proposed Framework, and Section VI Experimental Results, which are organized as follows.

II. RELATED WORK

In author proposed a novel framework for scheduling complexities of IoT environments that have mushroomed due to the widespread IoT devices and the resulting huge data generation [11]. To address this, they introduce edge computing as a complementary technology to cloud

computing, establishing an edge-cloud paradigm that fits the need of IoT applications. Although most studies focus on scheduling isolated tasks, some approaches discuss workflow scheduling interdependent task sequences in a more complex setting. Instead of presenting this as an NP-hard problem, the authors apply deep reinforcement learning (DRL) for workflow scheduling. Their methodology differs by using a hybrid actor crucial model that incorporates proximal policy optimization to handle complex scheduling requirements, as well as action space to distinguish between edge, cloud nodes. The comparison models report 56% improvements in energy consumption 46% improvements of execution time. A stable balance between energy efficiency and execution speed is reached. This finding emphasizes the framework ability to improve both energy efficiency and timeliness in completing tasks.

The telemedicine services in IoT fiber optics technology can improve the development and improve services through remote monitoring, communication between doctors and patients, and surgery [12]. Although researchers have developed a good amount of techniques in recent years, deep reinforcement learning approaches face limitations in the optimization of systems. In this regard, the authors design an adaptive security deep queue learning network for a fiber optics IoT healthcare system to manage the efficient scheduling of telemedicine applications. ASDQN uses different endoscopy datasets to control QoS constraints such as deadline, latency, and security. The aim of the paper is to minimize the makespan and maximize rewards across all nodes offloading and scheduling.

The healthcare monitoring systems, based on IoT-enabled devices, continuously collect, manage, and analyze patient data, helping in health monitoring [13]. The aim of optimizing the system, the authors design a fuzzy assisted ML framework incorporating fog computing, as well as increasing resource utilization efficiency by reducing healthcare monitoring effective resource costs. The framework uses the power of fuzzy logic in the fog computing system to optimize computing capacity on allocated fog and cloud resources. F-AMLF achieved great performance with accuracy at 93.6%, monitoring at 92.5%, prediction at 95.3%, and data management at 91.4%. It also lowered latency at 19.7%, energy usage at 20.1%, and cost at 21.5%, outperforming all other metrics.

The authors discuss the IoT edge devices and growing need for efficient task scheduling to reduce latency, computation time while ensuring high quality of service [6]. To that end, the authors introduce an advanced variant of the artificial hummingbird algorithm called CODA, which includes OBL, a confusion approach, and a DE. In CODA, DE optimizes chaotic maps, and OBL increases the initial population to improve the search without getting stuck in local optima. CODA is used for job scheduling in fog computing systems, with AHP for work prioritizing to reduce energy consumption and costs. The overall duration is also lowered by using

this approach. The authors compare CODA to 50 basic functions and evaluate it using simulations. CODA surpasses recognized meta-heuristics such as AHA, GSA, MFO, SOA, SSA, WOA, SCA, PSO, MVO, and DE by 46%, 8% cost savings, and 41% energy usage reduction. In this aspect, CODA is a more efficient job scheduler.

The authors focus on the use of cloud computing to manage user requests for resources and services in remote environments, particularly in healthcare systems that rely on smart IoT devices [14]. These devices create enormous volumes of data processed by medical sensors; thus, these devices need to be handled in an effective manner in terms of performance parameters such as throughput and latency. The authors propose a load balancing system called Resource Scheduling system for Cloud-Based Healthcare Environments to do this. The proposed framework uses GA, SARSA, and Q-Learning Reinforcement algorithms to predict optimal load management strategies. The proposed approach is built in MATLAB and is intended to improve energy efficiency while reducing delay. All of these are quantified and assessed using latency, makespan, and throughput metrics. The results reveal that the suggested mechanism outperforms conventional approaches, with high throughput and low makespan.

The authors present a framework for Internet of Medical Things applications that tries to address fog computing's latency and resource consumption problems [15]. As a result, they offer a modified particle swarm optimization technique to improve work scheduling and load balancing. They continue to use the implemented iFogSim toolkit to assess performance parameters like execution delay, cost, energy, and network consumption. Based on the experiments conducted, up to 20% latency improvements, up to 30% cost improvements in improved spectrum, and up to 15% improvements in both energy and network utilization were experienced. Moreover, the MPSO algorithm shows up to 80% improvements in resource utilization, which means it is suitable for IoMT applications in fog computing environments.

The current effort continues with the goal of offering an intelligent scheduling technique for FC systems using MODRL. A three-objective technique for selecting between fog and cloud node processing based on three criteria node load, distance, and job priority [16]. The authors address some of the major issues in job allocation and scheduling, and they back their technique with three separate deep reinforcement learning agents based on deep Q networks that reflect each of the objectives. To overcome the conflict generated by diverse objectives, the authors apply multi-objective optimization algorithms, specifically MOEA/D and NSGA2, to pick the ideal node. The simulation results demonstrate that all performance indicators such as job completion time, makepan, latencies, CPU load, and storage utilization.

The author formulated the TPDMAAC is a multi-agent actor-crucial framework based on transformer and policy

decoupling that was used to design a task offloading approach for vehicle fog computing [17]. It addresses the challenge arising from the heavy load imposed by many MVs when offloading tasks to edge servers, ES, whose processing duration is very high or low, especially for delay-sensitive tasks. The proposed model uses TLSFN for prediction and recasts actor-network with the help of a transformer-based TFE network and a policy decoupling network. With such self-attentive approaches, this approach of the proposed model helps TTDMAAC to respond well to variations in the size of inputs and offloading policies. Experimental results on real-world datasets show that the system reduces cost by 11.01%-12.03% compared to TPDMAAC algorithms and maintains the view of better adaptability to new environments.

The authors propose FETCH, a fog-enabled cloud computing framework designed address a problem in cloud-based infrastructures specifically related to health monitoring systems [18]. It integrates edge computing with deep learning to improve performance by reducing sensitivity to device latency in real-time applications such as heart disease monitoring. FETCH balances both accuracy of results with minimal response time, often conflicting goals with current fog computing paradigms. The framework is optimised for power consumption, network bandwidth, jitter, latency, and process execution time across a fogbus, demonstrating its usefulness in healthcare systems. This approach allows for more effective processing of huge amounts of data in sensitive applications such as health monitoring, which should increase the system adaptability and scalability.

In the work, the author conducts a comprehensive investigation of reinforcement learning and deep reinforcement learning methods to address such offloading difficulties in fog computing [19]. Fog computing was designed for resource-poor IoT devices and deals with the problems arising from long queues and associated delays when dealing with high request rates. Offloading tasks to fog or cloud layers can help improve performance and reduce latency; however, its efficient management in large-scale complex networks, especially wireless networks with complex state and action spaces, remains challenging. This paper classifies the types of fog computing offloading mechanisms into three types, such as value-based, policy-based, and hybrid-based algorithms, using RL and DRL. The comparison between these categories depends on key aspects such as offloading problem formulation, methods used, performance metrics, and evaluation tools. The strengths, weaknesses, and future research directions of RL and DRL are discussed along with their comments on how important a role both RL and DRL play in optimizing offloading decisions in such systems.

The author focuses on the problem of mapping periodic tasks in the HMPSoC architecture considering various constraints of performance, cost, energy, and reliability [20]. The mapping of tasks of HMPSoC architecture focuses on minimizing energy consumption, reducing the number of heterogeneous processors, and increasing reliability

within performance constraints using the ILP framework and multiple objective functions. Dynamic Voltage Scaling (DVS) is used to minimize energy consumption and task duplication is increases the reliability of the system. The feasibility of the approach is illustrated with experiments using several different task sets. Furthermore, two EDF-based heuristics are proposed to minimize energy consumption while satisfying performance and cost constraints. The results show that up to 62 percent energy savings can be achieved using the ILP-based method compared to when DVS is not applied. The heuristic methods also promise good results and are very close to the optimal results obtained from the ILP-based approach.

In this paper, the authors first propose a basic backup task assignment strategy called RELIEF [21]. Fog computing addresses real-time requirements by overcoming latency from cloud computing, however, reliability becomes a significant issue in executing real-time tasks, especially in dynamic environments where packet loss, power constraints, and external disturbances affect the completion of the task. The RELIEF technique use RL to identify appropriate nodes for primary and backup jobs; with these nodes, activities are completed on schedule, regardless of the challenges. The RL technique balances fog devices' communication delays with task distribution, boosting system reliability. Simulation results reveal that RELIEF outperforms prior approaches, reducing task dropping rates by up to 84%. This improves system stability by around 72%, making it an efficient solution for real-time work in fog computing situations.

The fog Computing and QoS related e-healthcare services improvement is proposed through low-latency data processing [22]. Fog reduces latency by performing some processing close to IoT devices to optimize performance while processing. The proposed model integrates Markov reinforcement learning with Monte Carlo learning for further improvement in data storage and service quality improvement. It selects the best nodes for fog to reduce latency during high-level health monitoring construction and allows for dynamic work allocation and load balancing. The implementation of the fog bus design has resulted in increased resource efficiency, bandwidth, jitter, latency, execution time, and accuracy.

The fog computing is a decentralized system that provides flexibility in resource placement between data-generating devices and the cloud to increase performance [23]. Due to the low resource and low latency requirements in fog computing, it is not possible to adopt new virtualization technologies such as virtual machine or container technology for task scheduling and resource management. LB has been extensively investigated in cloud computing, but not in fog computing due to the heterogeneity and vast number of devices. This work proposes an effective dynamic load balancing technique called CNN-based modified particle swarm optimization. EDLB includes three modules Fog Resource Monitor (FRM), CNN-based Classifier (CBC), and Optimized Dynamic Scheduler. The FRM module monitors server resources, the CBC module classifies servers as

suitable or unsuitable, and the ODS module assigns jobs to the most appropriate servers. Such a method provides continuous service by significantly reducing reaction times, hence increasing resource usage. This strategy is particularly effective for real-time systems, such as healthcare applications.

The growth of IoT applications, there is an enormous generation of data that cannot be processed on standalone IoT devices [24]. Offloading all computations to cloud data centers leads to issues such as high bandwidth consumption and energy consumption as well as latency. It overcomes these challenges by placing the cloud's services closer to the user. To optimize task scheduling and resource allocation, it models the offloading problem as an MDP, considering QoS factors in terms of latency, energy, task deadlines, and priority. Three DRL-based solutions are proposed to maximize resource utilization - experimental results show that this method achieves a task deadline satisfaction rate of 96.23%, which is an 8.25% improvement over the state-of-the-art method.

The widespread use of IoT, healthcare, smart cities, and autonomous vehicle applications rely on real-time data processing and analytics [25]. Cisco term the Fog refers to the combination of edge and cloud computing. We present an effective prediction and resource allocation mechanism for fog settings, with a focus on healthcare applications. EPRAM addresses the resource allocation problem in fog by combining it with prediction techniques. It includes three modules (i) Data Preprocessing Module (DPM), (ii) Resource Allocation Module (RAM), and (iii) Effective Prediction Module. EPM employs a probabilistic neural network (PNN) to forecast heart attacks and other events based on IoT data.

In author fog computing is about task scheduling, which lags behind its performance and customer satisfaction. The researchers propose a hybrid approach combining GWO and Heterogeneous Earliest Finishing Time HEFT to address this issue [26]. The improved GWO algorithm improves the task scheduling problems of makepan minimization and throughput improvement in fog computing environments. For benchmarking, it is used with the iFogSim toolkit. The results show that IGWOA outperforms existing algorithms such as AEOSA, HHO, PSO, and FA with up to 9.34% makepan reduction and 72.56% higher than other approaches on a dummy workload. On real datasets of NASA iPSC and HPC2N datasets, it reduces makepan by 6.89% and increases throughput by 62.4%, 52.8% and 41.6%. These results show the effectiveness of IGWOA in fog resource scheduling.

The smart scheduling model based on hybrid Moth Flame Optimization is presented for scheduling the tasks in Internet of Health Things in an e-healthcare environment [27]. HMFO ensures uniform resource allocation and also enhances the quality of service. The model for job scheduling is learned from the Google Cluster dataset and applied to CloudSim simulation for real-time scheduling. It measures performance in shifts of resource utilization, response time, and energy consumption. Hybrid HMFO represents higher response

rates, lower cost, and reduced run times compared to other methods.

The energy-efficient Internet of Medical Things to enable task scheduling interoperability for real-time healthcare applications [28]. This approach uses fog computing to address cloud latency with task scheduling, which ensures that crucial tasks reach their deadlines while balancing energy consumption. The architecture tracks cardiac health conditions using ECG sensors and sends the data via Bluetooth to an ESP32 microcontroller, which in turn sends the data to a Wi-Fi-enabled fog scheduler. Mapping tasks to fog nodes In fact, it maps tasks to fog nodes based on a weighting formula that considers both energy and latency. Simulation results based on iFogSim2 show that EEIoMT is better than CHTM, LBS, and FNPA models in terms of energy consumption, latency, and network utilization.

The author presents an artificial intelligence-based latency-aware job scheduling method for IoT applications in small-scale fog computing [29]. The suggested solution is based on artificial neural networks and can partition jobs between numerous edge servers for parallel execution. This strategy shortens scheduling times and meets service level targets. This technique is efficient because it achieves low latency and response time while consuming less energy. Performance studies against cutting-edge methods demonstrate that the new scheduling methodology is highly effective and efficient in small-scale fog settings.

The author proposes a novel architecture [30], in which the main idea is represented by the concept of Quartet Deep Learning, which rejects the traditional limitations of scalability and latency issues from edge computing devices and cloud computing frameworks, this paper envisions. A model for healthcare management delivery as a fog service through IoT devices is to efficiently manage patient data based on user needs. Unlike current fog models that focus on accuracy or response time, it establishes a way to balance both in real-world healthcare applications. The effectiveness of the provided system is measured in terms of resource utilization, network throughput, congestion, accuracy, and runtime in the case of the FogBus framework. The system can be configured to operate in different modes to optimize the quality of service and assess the accuracy a given fog computing.

III. METHODOLOGY

The Figure 1 IoT-fog-Cloud describes a hierarchical framework that represents resource management suitable for crucial healthcare applications. The proposed architecture enhances real-time response capabilities and ensures efficient data management. It is a network of multiple IoT devices, including diagnostic sensors, wearable health devices, and patient monitoring systems, connecting them to multiple access points. Access points operate at primary data collection points that collect data from various IoT devices, and then move towards higher layers of data processing in an efficient way. Access point data also reduces the likelihood of bottlenecks, as centralization at this layer allows data flow in a

TABLE 1. Summary of related work.

Ref	Proposed Work	Limitations	Simulation Tool	Parameters
[11]	Energy and Time Optimized Scheduling using Proximal Policy Optimization	Centralized operation no distributed scheduling support	CloudSim toolkit	Energy consumption, deadline hits
[12]	Fiber-optics IoT healthcare system using ASDQN	Resource constraints at local devices lacks integration with cost and advanced resource failure mechanisms	Socket programming-based simulation	Latency, deadlines, resource utilization, security metrics, reward maximization
[13]	Fuzzy-assisted machine learning framework (F-AMLF)	High dependency on fog nodes lacks scalability with an increasing number of patients and data volume	Python	Latency, energy consumption, accuracy ratio, prediction ratio
[14]	An efficient task scheduling in fog computing using advanced artificial hummingbird algorithm (CODA)	High computational complexity, initial population dependency, scalability issues with increasing IoT nodes	MATLAB R2018b	Makespan, cost, energy consumption, task length
[15]	Resource scheduling in cloud-based health communication using GA, SARSA, and Q-Learning algorithms	High initial computational cost, dependency on preconfigured system parameters	MATLAB R2022b	Throughput, latency, makespan, number of servers, energy efficiency
[16]	Task scheduling using Modified Particle Swarm Optimization (MPSO) for Internet of Medical Things (IoMT) applications	High dependency on fog nodes, limited scalability for large-scale dynamic tasks	iFogSim modeling toolkit	Execution delay, energy consumption, execution cost, network bandwidth usage, resource utilization
[17]	An Intelligent Scheduling in Fog Computing System using Multi-Objective Deep Reinforcement Learning (MODRL)	Challenges in scalability and high computational requirements for multi-objective optimization in large-scale networks	Python environment with TensorFlow	Task completion time, makespan, queuing delay, propagation delay, transmission delay, processing delay, computational delay
[18]	TPDMAAC	Requires large training datasets, lacks scalability for extreme dynamic environments, dependency on historical data for predictions	MATLAB and Python	Queuing delay, task drop rate, task completion rate, energy consumption, average system cost
[19]	FETCH (Fog Enabled Technique for Clinical Healthcare)	Dependency on historical workload data, limited to record-based datasets	FogBus framework	Latency, jitter, execution time, power consumption, network bandwidth
[20]	Offloading Mechanisms in Fog Computing using Reinforcement Learning and Deep Learning Algorithms	Complexity in decision-making for large-scale wireless networks, challenges in scalability and handling dynamic resource allocation	MATLAB	Latency, task execution time, resource utilization, energy consumption, offloading cost
[21]	Scheduling tasks on diverse MPSoCs while considering energy and reliability	High computational cost for ILP-based solutions, limited to small task sets for optimal results	linear programming packages	Energy consumption, reliability, task duplication percentage, deadline constraints
[22]	A Reinforcement-Learning-Based Real-Time Task Assignment Strategy for Fault-Tolerant Fog Computing	Dependency on primary backup strategy, challenges with scalability for highly dynamic IoT networks	Python environment	Task drop rate, workload balancing, average reliability, system delay, fault tolerance
[23]	Quality of service-based machine learning in fog computing networks for e-healthcare services and data storage system	Limited scalability for highly dynamic environments, dependency on accurate data for effective Markov and Carlo learning models	FogBus framework	latency, process execution time, network bandwidth, power consumption
[24]	Effective Dynamic Load Balancing (EDLB) using CNN and Modified Particle Swarm Optimization (MPSO)	Dependency on fog servers, requires robust mechanisms for handling server failures	MATLAB	Makespan, average resource utilization (ARU), load balancing level (LBL), total cost
[25]	QoS-Aware Task Offloading in Fog Environments Using Multi-Agent Deep Reinforcement Learning	High computational demand for multi-agent training, challenges in real-time adaptation to dynamic workloads	Python with TensorFlow	Task completion rate, average utility, queue waiting time, energy consumption, task drop rate
[26]	EPRAM	High dependency on specific healthcare datasets, requires real-time response adaptation for diverse IoT setups	iFogSim tool	Latency, makespan, average resource utilization (ARU), load balancing level (LBL), response time
[27]	Improved Grey Wolf Optimization Algorithm (IGWOA)	Challenges in handling extreme scalability, higher computational cost for larger task sizes	iFogSim toolkit	Makespan, throughput, execution time, resource utilization, latency, task drop rate
[28]	Task Scheduling with Hybrid Moth Flame Optimization and Deep Neural Network Algorithm for E-Healthcare Systems	High initial computational cost for hybrid optimization, dependency on specific dataset structure	CloudSim environment	Resource utilization, response time, energy consumption, packet delivery ratio
[29]	EEIoMT	Dependency on ECG-specific tasks and datasets, challenges in resource scalability for large-scale IoMT networks	iFogSim2 toolkit	Energy consumption, latency, network usage, CPU utilization, task prioritization
[30]	Latency-Aware Task Scheduling for IoT Applications Based on Artificial Intelligence	Limited scalability in large-scale environments, performance degradation due to unbalanced workloads in small-scale fog networks	Python	Latency, service level objective (SLO) violations, scheduling time, migration time, energy consumption
[31]	Ensemble Deep Learning-Based Fog Computing Framework for Real-Time Cardiac Health Analysis in IoT Applications	High computational cost for deep learning models, dependency on structured data formats	FogBus framework with Python	Accuracy, latency, throughput, energy efficiency, task completion rate

continuous mode with multiple sources. The aggregated data is sent to the end node layer, where task priority processing is performed. At the end node, each incoming data task is

ranked and prioritized based on its urgency and relevance to healthcare needs. High-priority tasks include urgent alerts or crucial status updates to patients, while lower-priority

tasks are queued for later attention. This step is prioritized because it filters timely healthcare data from general or background information, allowing crucial data to receive faster processing.

Forwarding to task scheduler in fog layer matches tasks with priority levels assigned to available virtual machines. Fog nodes are close to data sources and can meet real-time processing needs with minimal latency. This localized task scheduling reduces the burden on central cloud resources and enables quick responses to crucial healthcare scenarios where immediate insights improve patient outcomes. VMs in fog nodes process high-priority tasks immediately using a structured scheduling algorithm that maximizes computational resources in real-time. All non-crucial or idle data transfers occur to the cloud layer after all immediate tasks have been processed. Long-term storage of data and advanced analytics are available in the cloud, enabling processing of larger amounts of data than would be possible on available cloud resources. So, offloading saves the capacity of fog nodes, in which low-priority data is moved to a scalable infrastructure for storage and further analysis if predictive insight or trend analysis or further health-related research is required. The hybrid IoT-fog-cloud architecture makes the best use of the potential of each layer. Therefore, the fog layer, due to its priority-based task scheduling mechanism, enables quick responses to crucial applications, while the storage and analytical capabilities in the cloud layer provide deep, retrospective insights. In an integrated approach, the architecture provides a robust and efficient framework for managing the high volume and diversity of healthcare data for the real-time and long-term needs of modern healthcare systems.

A. TASK PRIORITIZATION

Task prioritizing across all linked sensors begins at the access point (AP), where the first round of prioritization occurs. The second level of prioritizing takes place at the end nodes (ENs), where jobs are prioritized across all connected access points. Each sensor stream transmits perceived data, which should be prioritized on the first level based on its cruciality. This process is repeated at the end nodes, where the access points forward the prioritized tasks to the end nodes, which then prioritize the tasks from all connected access points. To classify tasks, two key parameters are used, namely heart rate (HR) and initial ECG value (IECG). Based on these parameters, the initial priority of each task is determined. The first parameter, heart rate, is categorized into three levels. If the heart rate is greater than 120 or less than 60, a task is given high priority, indicating a potentially crucial situation. If the heart rate is between 80 and 120, it is assigned moderate priority. If the heart rate falls between 60 and 80, it is considered low priority. This is expressed as

$$HR = \begin{cases} \text{high,} & \text{if the heart rate is higher than 120 or } < 60 \\ \text{moderate,} & \text{if the heart rate is between 80 and 120} \\ \text{low,} & \text{if the heart rate is between 60 and 80} \end{cases} \quad (1)$$

The second parameter is the QT interval (QT_{lc}) derived from the initial ECG value (IECG). The QT interval is the duration of electrical activity in the heart from the start of the Q wave to the end of the T wave on an electrocardiogram. It refers to the time between the start of ventricular contraction and the end of ventricular relaxation. The QT interval varies with heart rate, shortening as heart rate increases, requiring adjustment for accurate comparison across different heart rates. This adjustment is achieved using the following equation

$$QT_{lc} = QT + 0.154 \times (1.00 - \frac{60}{HR}) \quad (2)$$

In this equation, RR and QT are measured values with an estimated regression slope of 0.154. The QT_{lc} values are divided into levels for priority assignment. For males, low priority is assigned if $QT_{lc} \geq 431$ and $QT_{lc} \leq 450$. For females, the low priority range is $QT_{lc} \geq 451$ and $QT_{lc} \leq 470$. Any QT_{lc} value outside these ranges is considered high priority. This priority can be expressed as:

$$IECG = \begin{cases} \text{low,} & \text{if male, } (QT_{lc} \geq 431) \text{ and } (QT_{lc} \leq 450) \\ \text{low,} & \text{if female, } (QT_{lc} \geq 451) \text{ and } (QT_{lc} \leq 470) \\ \text{high,} & \text{if male, } (QT_{lc} < 431) \text{ or } (QT_{lc} > 450) \\ \text{high,} & \text{if female, } (QT_{lc} < 451) \text{ or } (QT_{lc} > 470) \end{cases} \quad (3)$$

Based on the HR and IECG assessments, each task is assigned an overall priority. If either the HR or IECG priority is high, the task is marked as 'crucial', indicating that it requires immediate attention. If neither is high, the task is classified as 'normal', indicating that no immediate intervention is required. When tasks are received by a fog node (FNA), they arrive with their initial priority (INP). The FN calculates the size of each incoming task and sorts them based on their priority and size. Tasks are then assigned according to the current nodes resource availability. If the present node does not have optimal resources to handle the task, it is forwarded to another fog node (FN) for further processing. This approach ensures efficient handling of tasks, with the most crucial ones being prioritized and the necessary resources being allocated. This method optimizes task handling in fog computing environments, especially for crucial healthcare tasks.

IV. PROPOSED TASK SCHEDULER ARCHITECTURE

In Figure 2, we describe an architecture to improve the efficiency of task scheduling in fog computing. Incoming tasks are assigned specific priority levels, which govern how the system uses resources, prioritizing high-priority tasks. The main advantages of a fog environment are scalability, fault tolerance, latency, scalability, and reliability. These jobs are handled by nodes placed among IoT devices and cloud, which make quick decisions based on local processing based on the assigned priorities. A deep reinforcement learning approach called DDPG is at the heart of the

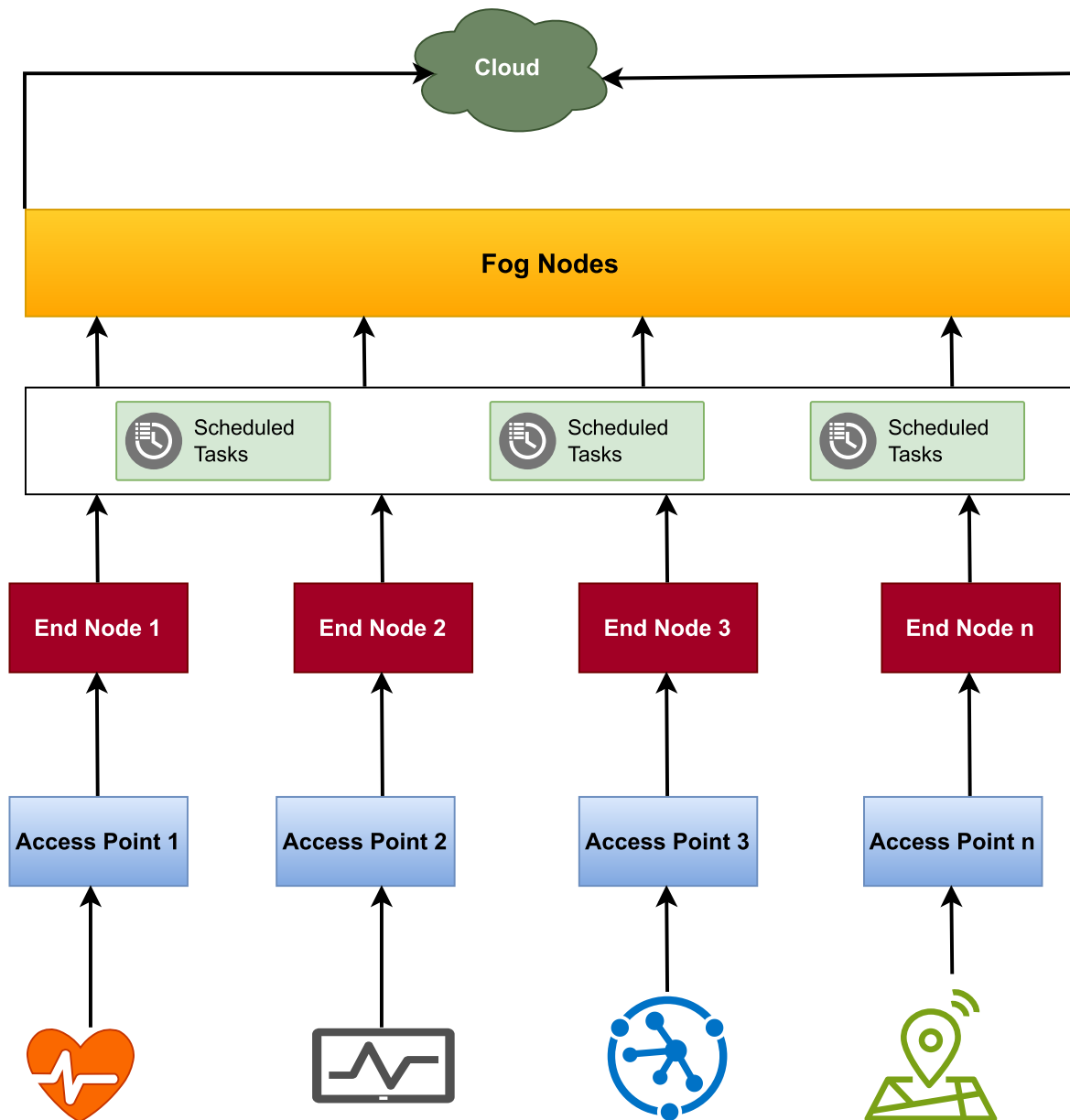


FIGURE 1. Proposed methodology.

scheduling mechanism and determines the optimal order and reassignment of computing tasks between fog nodes at runtime. DDPG application model allows it to track resource usage and adjust loads accordingly so that tasks are processed using available fog node capacity. Subsequently, the locally processed data is forwarded to the cloud for comprehensive, long-term storage, a process that may require intensive computing resources. The architecture in Shot enables seamless scheduling of tasks across the fog and cloud was applicable to crucial healthcare applications by optimizing response handling times in real time. In fog computing environments, ensuring resource availability is crucial to maintaining system capacity and performance. Resource availability is an assessment of whether a fog node has the resources required to effectively perform a given task.

Each fog node is equipped with various resources, including storage capacity, CPU units, RAM, and virtual machines (VMs), which must be evaluated to determine whether the node can meet the demands of the task.

Resource availability checking is a fundamental step in this process. It involves verifying that the fog node meets all the requirements for a task. In particular, the storage capacity of the available nodes must be sufficient to store the size of the tasks. This is determined by comparing the available nodes to the size of the tasks. In addition, the node must have enough CPU units to handle the task, which is checked by comparing the available CPU units to the CPU requirements of the tasks. RAM availability is similarly assessed by ensuring that the nodes RAM meets or exceeds the RAM requirements of the tasks. Finally, the node must have at least one VM to run

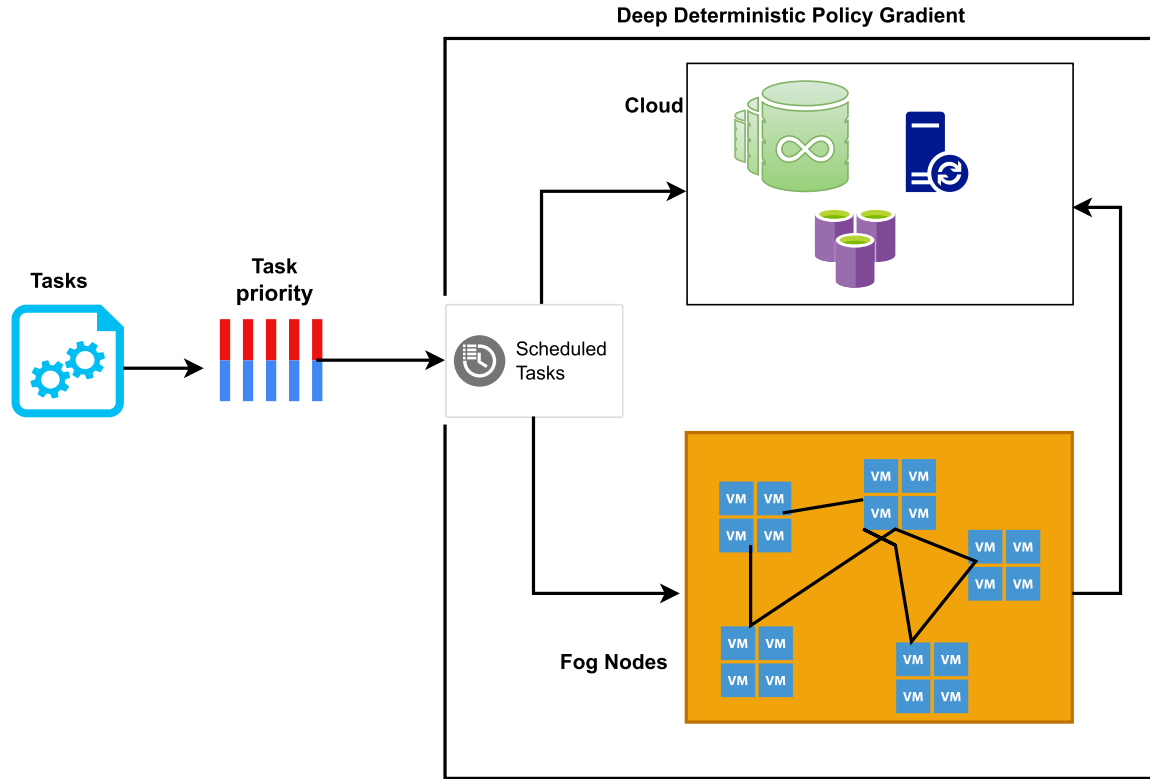


FIGURE 2. Proposed task scheduler architecture.

the task. These conditions are represented mathematically as follows

$$\text{Storage Check} = \text{available_resources} \geq T_{\text{size}} \quad (4)$$

$$\text{CPU Check} = \text{cpu_units} \geq c_{\text{pu_required}} \quad (5)$$

$$\text{RAM Check} = \text{ram} \geq r_{\text{am_required}} \quad (6)$$

$$\text{VM Check} = \text{vms} > 0 \quad (7)$$

If all these conditions are satisfied, the fog node is capable of handling the task. Otherwise, it is considered unsuitable for task. After a task was assigned fog node, the system adjusts the nodes resources accordingly. During task execution, the resources available to the nodes are temporarily reduced. This involves reducing storage capacity, CPU units, and RAM based on the needs of the tasks, as well as reducing the number of available VMs. These adjustments are calculated as follows

$$\begin{aligned} \text{Available Resources after assignment} \\ = \text{Available Resources} - T_{\text{size}} \end{aligned} \quad (8)$$

$$\begin{aligned} \text{CPU Units after assignment} \\ = \text{CPU Units before} - c_{\text{pu_required}} \end{aligned} \quad (9)$$

$$\begin{aligned} \text{RAM after assignment} \\ = \text{RAM before} - r_{\text{am_required}} \end{aligned} \quad (10)$$

$$\begin{aligned} \text{VMs after assignment} \\ = \text{VMs before} - 1 \end{aligned} \quad (11)$$

After the task is completed, resources are restored to their original level. This replenishment involves adding the task size back to storage, restoring CPU units and RAM, and increasing the number of VMs. Resource replenishment is calculated as follows

$$\begin{aligned} \text{Available Resources after completion} \\ = \text{Available Resources after assignment} + T_{\text{size}} \end{aligned} \quad (12)$$

$$\begin{aligned} \text{CPU Units after completion} \\ = \text{CPU Units after assignment} + c_{\text{pu_required}} \end{aligned} \quad (13)$$

$$\begin{aligned} \text{RAM after completion} \\ = \text{RAM after assignment} + r_{\text{am_required}} \end{aligned} \quad (14)$$

$$\begin{aligned} \text{VMs after completion} \\ = \text{VMs after assignment} + 1 \end{aligned} \quad (15)$$

By systematically managing these resources, the system ensures that fog nodes are used efficiently without being overloaded, thereby optimizing overall performance and reliability in a fog computing environment.

V. DEEP DETERMINISTIC POLICY GRADIENT (DDPG)

In fog computing, efficient scheduling is crucial in optimizing the distribution of tasks across fog nodes, improving system efficiency, and resource utilization. The process begins by deciding which fog node should handle each task. This decision is guided by a deep deterministic policy gradient algorithm that estimates the current state of each fog node

Algorithm 1 Determining the Initial Priority (IP)

```

1: Input: Collected tasks  $T$  from connected sensors
2: Output: Assigned initial priority for each task
3: for each task  $T$  do
4:   1.1 Evaluate Heart Rate (HR) Priority:
5:   if  $T_{HR} > 120$  or  $T_{HR} < 60$  then
6:      $INC \leftarrow \text{high}$ 
7:   else if  $80 < T_{HR} \leq 120$  then
8:      $INC \leftarrow \text{moderate}$ 
9:   else
10:     $INC \leftarrow \text{low}$ 
11:   end if
12:   1.2 Assess Initial ECG Value (IECG):
13:   if the patient is male then
14:     if  $431 \leq T_{QTc} \leq 450$  then
15:        $IECG \leftarrow \text{Low}$ 
16:     else
17:        $IECG \leftarrow \text{High}$ 
18:     end if
19:   else if the patient is female then
20:     if  $451 \leq T_{QTc} \leq 470$  then
21:        $IECG \leftarrow \text{Low}$ 
22:     else
23:        $IECG \leftarrow \text{High}$ 
24:     end if
25:   end if
26:   Assign  $T_{IECG}[i] \leftarrow IECG$ 
27:   1.3 Compute Initial Priority (INP):
28:    $INP \leftarrow T_{HR} + IECG$ 
29:   1.4 Update Priority List:
30:   Add or update the task in the priority list based on
   the computed  $INP$ 
31:   1.5 Forward to Fog Node (FN):
32:   Send the prioritized task to the suitable fog node for
   processing
33: end for

```

Algorithm 2 Evaluate Resource Feasibility

```

1: function CHECK_RESOURCES(node, task)
2:   if node.available_resources < task.size then
3:     return False
4:   end if
5:   if node.cpu_capacity < task.cpu_demand then
6:     return False
7:   end if
8:   if node.memory < task.memory_demand then
9:     return False
10:  end if
11:  if node.active_vms  $\leq 0$  then
12:    return False
13:  end if
14:  return True
15: end function

```

factoring in metrics such as available storage, CPU units, RAM, and the number of VMs. The DDPG agent processes this state information to select the most suitable fog node for the task. The availability of each fog node resource is presented as a vector. The DDPG agent generates an action based on this state, which is a continuous value representing the selected node. This action is scaled and hardened to ensure that it maps to a valid fog node index. For example, if the DDPG agent outputs the action value a , it is converted to the node index n using

$$n = \text{round}(\text{clip}(a, -1, 1) \times (\text{Number of Nodes} - 1)) \quad (16)$$

Once the selected node is identified, the system checks whether there are sufficient resources to execute the task. This involves comparing the available resources with the tasks' requirements. For example, the available storage available_resources is the task size T_{size} :

$$\text{Storage Check} = \text{available_resources} \geq T_{\text{size}} \quad (17)$$

$$\text{CPU Check} = \text{cpu_units} \geq \text{cpu_required} \quad (18)$$

$$\text{RAM Check} = \text{ram} \geq \text{ram_required} \quad (19)$$

Equation (21) indicates the highest level of cruciality. A patient's cruciality is classified as "high" if any of the following conditions are present: high heart rate, high blood pressure, or an initial ECG shows high urgency. Mathematically, this can be expressed as

$$\begin{aligned} \text{cruciality} \\ = \text{High if } (HR = \text{High OR } BP = \text{High OR } FECG = \text{High}) \end{aligned} \quad (20)$$

Equation (21) is used when the conditions for high severity are not met but moderate levels are observed. If heart rate, blood pressure, or initial ECG are classified as moderate and none of the parameters are high, the severity is considered "moderate".

$$\begin{aligned} \text{cruciality} = \text{Moderate if} \\ (HR \neq \text{High AND} \\ BP \neq \text{High AND} \\ FECG \neq \text{High}) \\ \text{AND} \\ (HR = \text{Moderate OR} \\ BP = \text{Moderate OR} \\ FECG = \text{Moderate}) \end{aligned} \quad (21)$$

Equation (22) represents a scenario where none of the high or medium conditions are met. If all parameters heart rate, blood pressure, and initial ECG are classified as low, the severity is considered "low". This situation is described

$$\begin{aligned} \text{cruciality} = \text{Low if} \\ (HR \neq \text{High AND} \\ BP \neq \text{High AND} \end{aligned}$$

Algorithm 3 Assign Task

```

1: function ASSIGN_TASK(fog_node, task)
2:   if CALCULATE_RESOURCE_AVAILABILITY(fog_node, task) then
3:     // Decrement resources
4:     fog_node.available_resources  $\leftarrow$  fog_node.available_resources - task.Tsize
5:     fog_node.cpu_units  $\leftarrow$  fog_node.cpu_units - task.cpu_required
6:     fog_node.ram  $\leftarrow$  fog_node.ram - task.ram_required
7:     fog_node.vms  $\leftarrow$  fog_node.vms - 1
8:     // Simulate task execution
9:     SIMULATE_TASK_EXECUTION(task.average_execution_time)
10:    // Replenish resources
11:    fog_node.available_resources  $\leftarrow$  fog_node.available_resources + task.Tsize
12:    fog_node.cpu_units  $\leftarrow$  fog_node.cpu_units + task.cpu_required
13:    fog_node.ram  $\leftarrow$  fog_node.ram + task.ram_required
14:    fog_node.vms  $\leftarrow$  fog_node.vms + 1
15:    return True
16:   else
17:     return False
18:   end if
19: end function

```

Algorithm 4 Load Balancing

```

1: function ASSIGN_TASKS_WITH_DDPG(env, agent, fog_nodes, tasks, metrics)
2:   for each task in tasks do
3:     // Gather state information of all fog nodes
4:     state  $\leftarrow$  GET_CURRENT_STATE(fog_nodes)
5:     // Use DDPG agent
6:     action  $\leftarrow$  AGENT.POLICY(state)
7:     // Map action to the nearest fog node
8:     selected_node  $\leftarrow$  round (clamp(action, -1, 1)  $\times$  (len(fog_nodes) - 1))
9:     // Check if the selected node can handle the task
10:    if CALCULATE_RESOURCE_AVAILABILITY(fog_nodes[selected_node], task) then
11:      // Assign task to the selected node ASSIGN_TASK(fog_nodes[selected_node], task)
12:    else
13:      // Forward task to backup or alternative node FORWARD_TASK_TO_BACKUP(task)
14:      metrics['fault_tolerance']  $\leftarrow$  metrics['fault_tolerance'] + 1
15:    end if
16:    // Update performance metrics
17:    metrics['total_delay']  $\leftarrow$  metrics['total_delay'] + CALCULATE_TASK_DELAY(task)
18:    metrics['makespan']  $\leftarrow$  env.now
19:   end for
20: end function

```

$$\begin{aligned}
 & FECC \neq \text{High}) \\
 & \text{AND} \\
 & (HR = \text{Moderate AND} \\
 & \quad BP = \text{Moderate AND} \\
 & \quad FECC = \text{Moderate}) \quad (22)
 \end{aligned}$$

Through these equations, the severity of each patient's condition is effectively categorized, ensuring that those with the most urgent needs receive prompt attention. This systematic approach helps prioritize medical responses and

optimize patient care. The prioritization mechanism in the proposed framework classifies tasks as high, medium, or low priority based on heart rate (HR) and initial ECG (IECG) values. Heart rate exceeding 120 beats per minute (tachycardia) or falling below 60 beats per minute (bradycardia) often indicates a critical or abnormal condition requiring immediate medical attention. Studies in cardiology confirm that such extremes in heart rate are associated with heightened risks, such as arrhythmias, cardiac arrest, or other life-threatening events. These thresholds are commonly used in emergency medicine to trigger rapid responses. The QT

interval correction (QTlc) is a well-established parameter in assessing cardiac function. Abnormal QT values (e.g., QTlc > 450 ms in males and QTlc > 470 ms in females) indicate risks of ventricular arrhythmias and sudden cardiac events, making them high-priority conditions. Such thresholds are derived from clinical studies and widely adopted in medical practice. By incorporating these thresholds, the proposed prioritization mechanism aligns with clinically relevant parameters. Tasks associated with abnormal HR or IECG values are classified as high priority to ensure immediate processing in life-critical healthcare applications.

Tasks can be categorized as critical or non-critical based on their impact on the overall system. High-crucial tasks typically have tighter deadlines or greater consequences if delayed, while low-crucial tasks can be scheduled with more flexibility. Identifying and prioritizing task cruciality is key to optimizing system performance, ensuring that high-priority tasks receive the necessary resources and attention to meet deadlines without compromising the efficiency of non-critical tasks. Deep Deterministic Policy Gradient (DDPG) is an effective reinforcement learning algorithm for optimizing task scheduling in dynamic environments. By learning from past experiences, DDPG minimizes key performance metrics like makespan, energy consumption, trust, and fault tolerance. The actor in the DDPG framework dynamically allocates tasks to resources in a way that reduces the overall completion time (makespan), ensures reliable execution (trust), and handles failures through redundancy and reallocation (fault tolerance). Additionally, DDPG can optimize energy use by scheduling tasks on power-efficient resources, leading to lower energy consumption. Overall, DDPG adapts to various system constraints, improving efficiency and robustness in task scheduling.

Makespan, defined as the total time taken to complete all tasks, is a critical metric in fog computing, especially for healthcare applications where timely processing is essential. The Deep Reinforcement Learning (DRL) model minimizes makespan by dynamically allocating tasks to fog nodes based on their real-time resource availability, such as CPU, RAM, and storage capacity. Unlike traditional scheduling, which often follows predefined static rules, DRL continuously learns from system states to optimize task distribution. Fog nodes inherently reduce latency by bringing processing closer to the data source, may occur if tasks are not distributed effectively.

$$\text{Makespan} = \max_i (\text{Finish Time of Task}_i) \quad (23)$$

Energy consumption is a significant concern in fog computing environments due to the resource constraints of edge devices. The DRL algorithm optimizes energy usage by intelligently scheduling tasks to nodes with the most efficient energy profiles. It avoids assigning low-priority tasks to high-power nodes, thereby reducing unnecessary energy expenditure.

$$E = P \cdot t \quad (24)$$

where P is the power consumption of the node, and t is the task execution time.

Trust in fog computing refers to the reliability of nodes to execute tasks accurately and securely. The DRL framework improves trust by prioritizing task allocation to nodes with higher historical success rates and lower failure occurrences. While fog nodes inherently decentralize processing and increase system reliability, DRL further enhances trust by dynamically evaluating and selecting nodes based on their reliability and performance.

$$\text{Trust} = \frac{\text{Successful Tasks}}{\text{Total Tasks}} \quad (25)$$

A higher trust value indicates better system reliability. DRL ensures that tasks are consistently executed by dependable nodes, reducing the probability of failures or delays. This capability is particularly critical for healthcare applications, where task failures can directly impact on results.

Fault tolerance is critical in ensuring system reliability, especially in healthcare applications where task interruptions can have severe consequences. The DRL framework enhances fault tolerance by dynamically reallocating tasks when a node fails or becomes overloaded. This is achieved by continuously monitoring node availability and resource utilization.

$$\text{Fault Tolerance} = \frac{\text{Rerouted Tasks}}{\text{Total Tasks}} \quad (26)$$

Lower fault tolerance values indicate fewer rerouted tasks and better system stability. While fog nodes inherently improve fault tolerance through decentralized processing, the DRL model ensures seamless task reallocation, minimizing the impact of node failures.

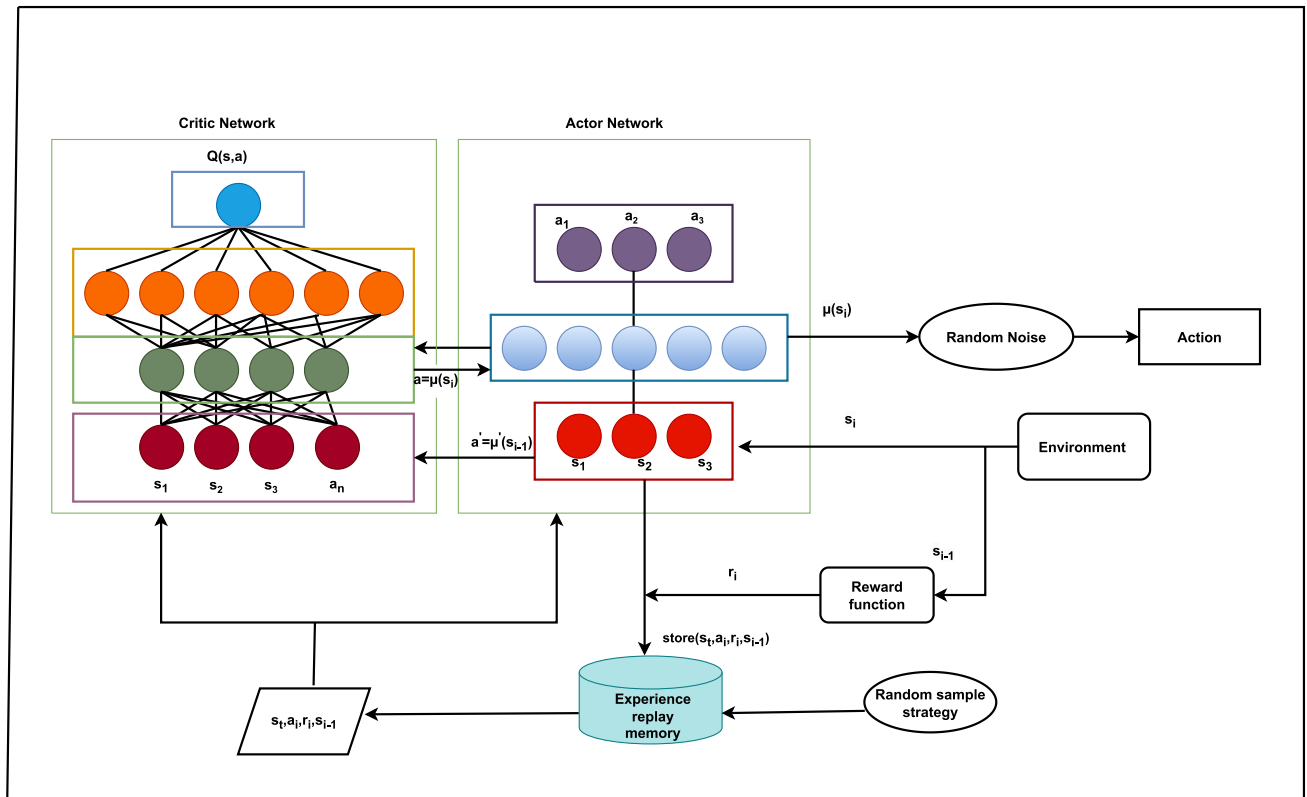
DDPG (Deep Deterministic Policy Gradient) Figure 3 is a model-free, off-policy reinforcement learning method that works for continuous action spaces. Since the core of this method is inspired by the optimal combination of policy-based and value-based reinforcement learning methods, DDPG is particularly useful in complex decision-making scenarios that require partial precision and openness to change. DDPG takes advantage of two neural networks actor network, critic network, to allow agent to learn optimal policies by determining the actions that the actor should take and the value that the critic has for these actions. The DDPG algorithm selects actions depending on the present condition of the environment and seeks to maximize cumulative rewards. The critic network criticizes these behaviors, estimating the expected future rewards or Q-values connected with them to prevent loss of consistency during training, DDPG applies the use of target networks for the actor as well as for the critic, which gradually updates the targets as they age and ensures that training is smooth. Another aspect of experiencing replay is how past experiences are stored, allowing them to be randomly sampled to further disrupt the correlation. In this study we propose task scheduling using DDPG.

Algorithm 5 Calculate Task Cruciality

```

1: function CALCULATE_TASK_CRUCIALITY(task)
2:    $hr\_priority \leftarrow (task['heart\_rate\_priority'] == 'high')$ 
3:    $bp\_priority \leftarrow (task['blood\_pressure\_priority'] == 'high')$ 
4:    $ecg\_priority \leftarrow (task['ecg\_priority'] == 'high')$ 
5:   if  $hr\_priority \vee bp\_priority \vee ecg\_priority$  then
6:     return 'High'
7:   else if  $\neg hr\_priority \wedge \neg bp\_priority \wedge \neg ecg\_priority$ 
8:      $\wedge (task['heart\_rate\_priority'] == 'moderate' \vee$ 
9:      $task['blood\_pressure\_priority'] == 'moderate' \vee$ 
10:     $task['ecg\_priority'] == 'moderate')$  then
11:     return 'Moderate'
12:   else
13:     return 'Low'
14:   end if
15: end function

```

**FIGURE 3.** DDPG architecture.**A. STATE**

DDPG is a state representation for task scheduling. It contains key features of fog nodes that influence scheduling decisions. Each fog node is represented by a state vector s , which contains available resources (storage capacity in GB), CPU units (number of processing units), RAM (amount of memory in GB), and VMs (number of virtual machines). This vector, captures a comprehensive snapshot of each node's resource availability, providing a foundation for making informed

scheduling choices.

$$s = [\text{available_resources}, \text{cpu_units}, \text{ram}, \text{vms}] \quad (27)$$

B. ACTION

The action selection process in DDPG involves an actor network that mapping the state vector s to continuous action a . In task scheduling, this action typically corresponds to selecting an appropriate fog node for task assignment. The

actor network's policy function $\mu(s)$ generates an action a that guides the task assignment with the goal of optimizing performance based on current knowledge. After an action is selected, the action evaluation phase evaluates its quality by the critic network. The critic values the selected action by estimating a Q-value $Q(s, a)$, which represents the expected future rewards of performing s in state a . This evaluation helps determine whether the action is consistent with the optimal scheduling goals, guiding the actor network to improve its approach.

C. REWARDS

The reward calculation evaluates the effectiveness of a scheduling action. The reward function r is designed to encourage desirable efficient task time, balanced resource utilization, efficient load distribution. The reward function can incorporate factors such as task completion time, resource efficiency, and load balancing, guiding the learning process in favor of beneficial scheduling strategies.

$$\text{Reward}(s, a) \quad (28)$$

D. ACTOR CRITIC NETWORK

To train the critic network, the current reward is combined with a discounted estimate of future rewards to determine the target value. The target Q-value (y) is computed as follows

$$y = r + (1 - d) \cdot \gamma \cdot Q_{\text{target}}(s', a') \quad (29)$$

r is the immediate reward, d is the done flag signaling that the episode is complete, γ is the discounted item, s' is the next state, and a' is the target actor network's suggested action. This target value aids critic network learning by offering a realistic estimate of future rewards. During the model update phase, the critic network is enhanced by reducing the mean squared error (MSE) between the estimated and target Q-values. The crucial loss function is expressed as follows

$$\text{crucial loss} = \text{MSE}(y, Q(s, a)) = \mathbb{E}[(y - Q(s, a))^2] \quad (30)$$

The actor network is updated to maximize the Q-value of its actions, which gives the actor a loss function:

$$\text{Actor Loss} = -\mathbb{E}[Q(s, \mu(s))] \quad (31)$$

This dual strategy, which optimizes both the critic and actor networks, ensures that the policy and value functions adapt to increase the efficiency of task scheduling. Soft updates are used to upgrade the target networks and assure their stability. This procedure entails merging the weights of the learned and target networks using

$$\theta_{\text{target}} \leftarrow \tau \cdot \theta + (1 - \tau) \cdot \theta_{\text{target}} \quad (32)$$

where θ denotes the weights of the learned networks, θ_{target} denotes the target network weights, and τ is the small update rate. Finally, the training process for DDPG involves iteratively updating the crucial network to reduce its loss, improving the actor network to increase Q-values, and periodically updating the target networks to maintain

stability. This iterative refinement process optimizes the ability of the DDPG algorithm to efficiently allocate fog nodes, balance resource usage, and optimize performance in fog computing environments.

TABLE 2. Table of mathematical notations.

Notation	Description
T_{size}	Task size
cpu_units	CPU units required for a task
ram_required	RAM required for a task
vms	Number of vm required for task execution
available_resources	storage, CPU, RAM, VMs on a fog node
cpu_required	CPU requirements for executing a task
n	Node index derived from DDPG action output
$Q(s, a)$	Q-value function
s	State vector representing a fog node resource status
a	Action determined by the policy in DDPG
r	Reward
γ	Discount factor
y	Target Q-value for training the critic network
τ	Soft update rate for updating target networks
θ	Weights of the learned networks in DDPG
θ_{target}	Weights of the target networks in DDPG
d	Done flag indicating episode completion in reinforcement learning
s'	Subsequent state after action a is taken
a'	Action and target actor network
B	Batch size for training
L_Q	Loss function for the critic
L_π	Loss function for the actor
R	Replay buffer

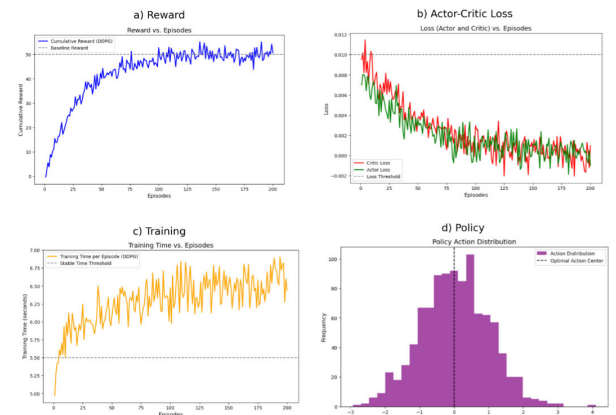


FIGURE 4. DDPG model results.

1) EVALUATION METRICS OF DDPG MODEL

The Figure Figure 4 and its training episodes discuss how the DDPG algorithm improves task scheduling in healthcare-centric systems. The reward graph emphasizes that the model

Algorithm 6 Deep Deterministic Policy Gradient (DDPG) Algorithm

```

1: Initialize the policy network  $\mu(s|\phi^\mu)$  with parameters  $\phi^\mu$ 
2: Initialize the value network  $V(s, a|\phi^V)$  with parameters  $\phi^V$ 
3: Initialize the target policy network  $\mu'$  with parameters  $\phi^{\mu'} \leftarrow \phi^\mu$ 
4: Initialize the target value network  $V'$  with parameters  $\phi^{V'} \leftarrow \phi^V$ 
5: Set up experience buffer  $E$  with size  $N$ 
6: for each episode  $k$  do
7:   Reset the environment and obtain the initial state  $s_0$ 
8:   for each step  $t$  within the episode do
9:     Action Selection:
10:    Choose action  $a_t \leftarrow \mu(s_t|\phi^\mu) + \epsilon$ , where  $\epsilon$  represents noise
11:    Action Execution:
12:    Perform action  $a_t$  and observe  $r_t$  and  $s_{t+1}$ 
13:    Store Experience:
14:    Save transition  $(s_t, a_t, r_t, s_{t+1})$  in  $E$ 
15:    Mini-Batch Sampling:
16:    Draw a random mini-batch  $\{(s_j, a_j, r_j, s_{j+1})\}$  from  $E$ 
17:    for each transition in the mini-batch do
18:      Compute Target:
19:       $z_j \leftarrow r_j + \gamma \cdot V'(s_{j+1}, \mu'(s_{j+1}|\phi^{\mu'})|\phi^{V'})$ 
20:      Update Value Network:
21:      Compute loss:  $\mathcal{L}_V = \frac{1}{M} \sum (z_j - V(s_j, a_j|\phi^V))^2$ 
22:      Optimize  $\phi^V$  by minimizing  $\mathcal{L}_V$ 
23:    end for
24:    Update Policy Network:
25:    Calculate policy loss:  $\mathcal{L}_\mu = -\frac{1}{M} \sum V(s_j, \mu(s_j|\phi^\mu)|\phi^V)$ 
26:    Optimize  $\phi^\mu$  by minimizing  $\mathcal{L}_\mu$ 
27:    Update Target Networks:
28:     $\phi^{\mu'} \leftarrow \tau \cdot \phi^\mu + (1 - \tau) \cdot \phi^{\mu'}$ 
29:     $\phi^{V'} \leftarrow \tau \cdot \phi^V + (1 - \tau) \cdot \phi^{V'}$ 
30:  end for
31: end for

```

learns to optimize task scheduling as the reward function is gradually maximized, such goals might be to minimize makespan and focus on critical tasks with abnormal heart rate or ECG values. In the early stages, the reward oscillates with all possible strategies of the model but converges to an approximately optimal scheduling policy after a few time steps. The actor-critic loss graph shows how well the actor and critic networks improve as the loss values decrease, indicating that the model becomes more effective at predicting and acting on scheduling decisions. This stabilization loss reflects the model's ability to effectively prioritize tasks, such as performing emergency patient monitoring, while avoiding resource congestion. The training graph shows the computational effort required to train the model, which initially increases as the algorithm explores different strategies, but eventually stabilizes, reflecting improved efficiency and scalability for real-time healthcare task scheduling. The policy graph shows how the DDPG algorithm dynamically distributes tasks across fog nodes to ensure balanced resource utilization. This is particularly important in healthcare systems, where timely assignment of critical tasks and avoidance of node congestion are essential

for system stability and fault tolerance. The above graph and analysis show that the DDPG algorithm performs well in an efficient, scalable, and reliable manner for real-time decision making in healthcare settings.

VI. EXPERIMENTAL RESULTS

In this experiment, we used the SimPy simulation environment, which runs on the robust platform of the Windows 11 operating system and is powered by an Intel Core i9 processor, 32GB of RAM. We classified our dataset for the experiments into three different scales: small, medium, and large. This classification is based on a fine-grained description of the specific characteristics of each task, such as priority level of each task amount of resources (memory used by applications, CPU, and network usage), and strict deadlines. The complexity of the tasks and the demands on computing resources. We also recorded key performance indicators such as free storage, used CPU, and available network bandwidth from the fog nodes, and trained the proposed DDPG for 200 episodes. Our experimental analysis is extensive, covering many key performance metrics that are important for assessing the efficiency of

distributed computing systems. These include Makespan, which measures the total time described to complete all tasks, energy consumption, which estimates the energy used in task execution, fault tolerance, which estimates the ability to dynamically reroute tasks to handle task failures and maintain system integrity, and trust. It assesses the reliability of task execution and the reliability of nodes in the network. Comprehensive performance analysis on three advanced computational models: LSTM, DQN, A2C. Each model was then tested against metrics to determine its ability to handle and execute tasks in simulated of fog computing.

A. DATASET

The dataset in this research [31] is designed to simulate task scheduling scenarios in fog computing especially for crucial health applications. It has features that reflect real-world resource constraints, task priorities, and system dynamics, allowing for robust testing and validation of scheduling algorithms. The dataset was carefully constructed to eliminate anomalies and incomplete records, thereby making the results reliable. The dataset is task-dependent, including priority levels, required resources such as memory and CPU usage, and bandwidth availability along with specific execution deadlines. Such parameters are crucial for modeling different scenarios when fog nodes are undertaking tasks with high priority levels and computational intensity. Furthermore, additional features are included to indicate the status of fog nodes, for example, available storage, CPU cores being used, and network bandwidth. For example, crucial healthcare tasks are given high priority due to their urgency to process, while routine tasks are assigned to less busy nodes or pushed to cloud resources. The dataset also includes latency, energy consumption, and trust parameters, which are key to addressing the efficiency and reliability of task scheduling mechanisms. To validate the proposed methodologies, the dataset is divided into small, medium, and large task scenarios for comprehensive analysis under different levels of workload intensity. The structured approach ensures that the scheduling algorithms are robust, scalable, and adaptable to the dynamic conditions of fog computing environments.

B. MAKESPAN

Makespan is a key performance metric in timely processing in fog computing environments, especially in healthcare systems. It calculates the total time required to do all jobs during a given scheduling cycle. Effective Makespan management improves resource utilization, reduces latency, and improves system responsiveness. Optimizing MakeSpan healthcare fog computing systems ensures faster data processing, efficient workload distribution, and improved service delivery for crucial applications such as patient monitoring and emergency response. Implementing an advanced scheduling algorithm such as our proposed work can significantly reduce MakeSpan and balancing task priority and system performance. MakeSpan experimental configurations were evaluated and the dataset was divided into three groups: small

(dataset 100 to 350), medium (dataset 400 to 650), and large (dataset 700 to 1000). The results for small datasets are shown in Table 3, where our proposed DDPG algorithm is compared with the baseline algorithms LSTM, DQN and A2C. For a dataset of 100 to 350 tasks, the proposed DDPG reduces the Makespan when compared with the existing algorithms. The comparison of the baseline algorithms and the proposed work is clearly shown in Figure 5. The results for medium datasets are shown in Table 4, where our proposed DDPG algorithm is compared with the baseline algorithms LSTM, DQN and A2C. For datasets of 400 to 650 tasks, the proposed DDPG reduces the MakeSpan when compared with the existing algorithms. The comparison of existing algorithms and the proposed work is clearly shown in Figure 6. The results for large datasets are shown in table 5. Our proposed DDPG algorithm is compared with LSTM, DQN and A2C baseline methods. For datasets of tasks ranging from 700 to 1000, the proposed DDPG reduces the makespan compared to the current algorithms. The contrast of existing algorithms and the proposed approach is well demonstrated in Figure 7.

TABLE 3. Makespan for small datasets.

Datasets	LSTM	DQN	A2C	DDPG
100	702.5	675.5	660.4	648.9
150	835.6	810.9	795.6	784.5
200	915.4	890.7	875.5	857.9
250	980.2	955.8	940.3	923.4
300	1045.8	1020.5	1005.6	984.8
350	1110.5	1085.8	1070.4	1044.7

TABLE 4. Makespan for medium datasets.

Datasets	LSTM	DQN	A2C	DDPG
400	1162.5	1148.8	1125.9	1109.2
450	1356.7	1318.4	1280.5	1261.4
500	1415.4	1395.8	1375.4	1358.1
550	1486.6	1465.7	1445.6	1412.9
600	1568.2	1545.5	1525.8	1495.7
650	1689.8	1640.6	1620.6	1583.7

TABLE 5. Makespan for large datasets.

Datasets	LSTM	DQN	A2C	DDPG
700	1728.7	1687.8	1689.8	1635.4
750	1792.8	1764.7	1745.6	1709.2
800	1865.4	1845.6	1825.4	1789.7
850	1954.5	1937.5	1910.6	1872.1
900	2029.8	2009.6	1985.5	1947.4
950	2139.6	2124.8	2090.8	2053.9
1000	2294.8	2256.6	2192.6	2139.5

C. ENERGY CONSUMPTION

Energy consumption metric is key to maintaining efficient energy consumption operations and reducing costs in fog computing system, especially in healthcare applications. It measures the total energy consumed during task execution. Reducing energy consumption ensures device operation,

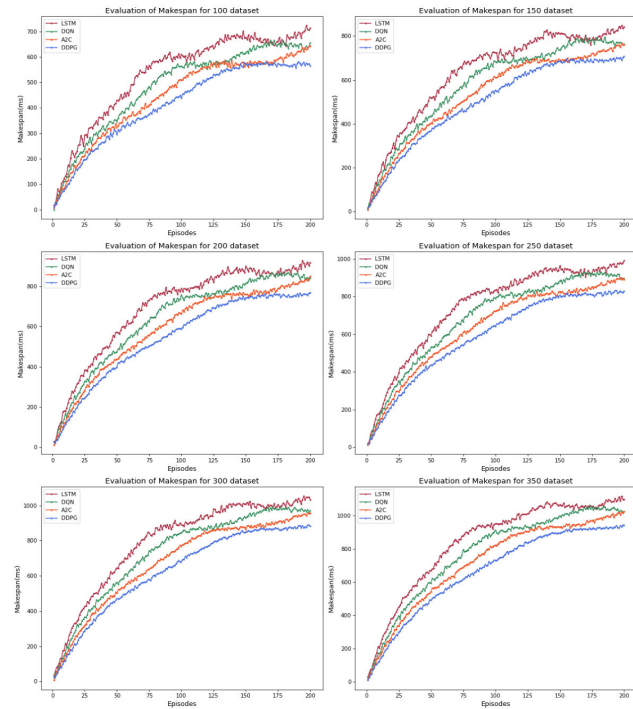


FIGURE 5. Makespan of 100_350 datasets.

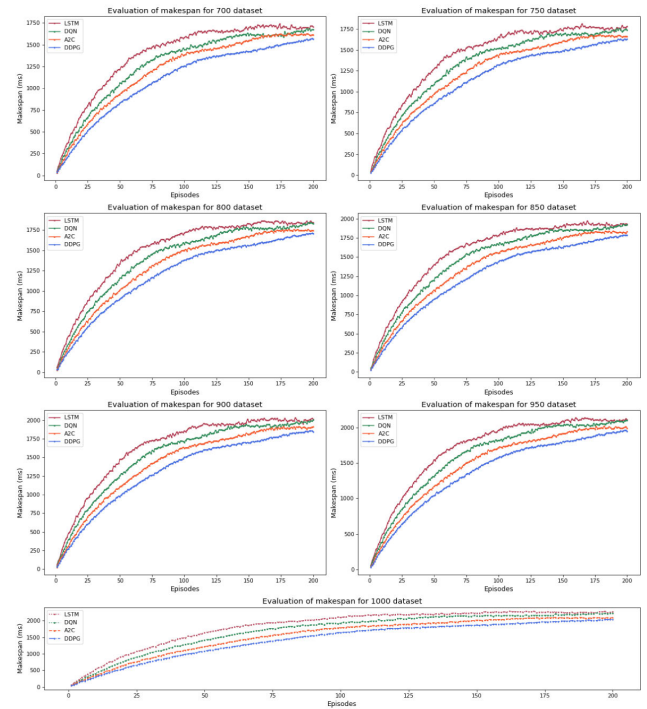


FIGURE 7. Makespan of 700_1000 datasets.

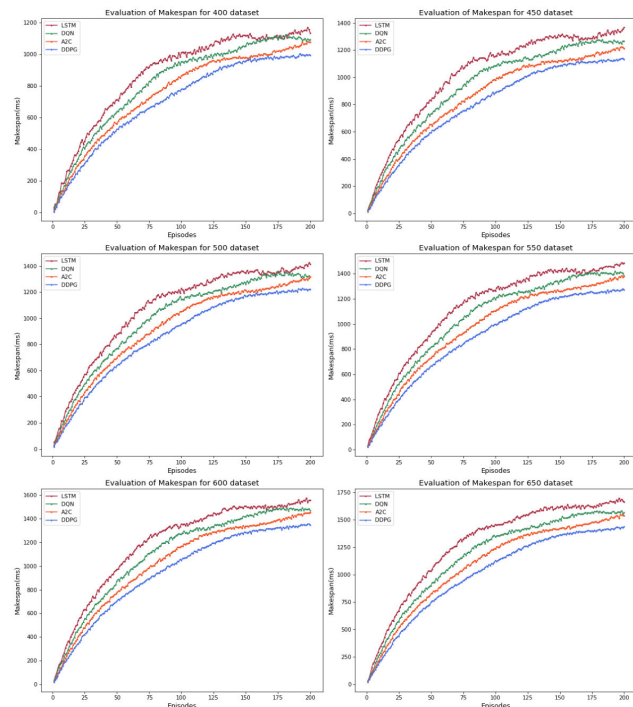


FIGURE 6. Makespan of 400_650 datasets.

reduced environmental impact, and cost effectiveness, especially in resource-constrained fog nodes. Energy efficient scheduling algorithm like the proposed systems optimizes resource allocation, balances workloads, and reduces unnecessary energy waste. This leads to improved system stability

and better performance in crucial healthcare environments, ensuring reliable and responsive care delivery. The energy consumption experimental configuration was evaluated and the dataset was divided into three groups small (dataset 100 to 350), medium (dataset 400 to 650), and large (dataset 700 to 1000). The results for small datasets are shown in Table 6, where our proposed DDPG algorithm is compared with the baseline algorithms LSTM, DQN and A2C. For a dataset of 100 to 350 tasks, the proposed work has reduced the energy consumption of DDPG when compared with the baseline algorithms. The comparison the baseline algorithms and the proposed work is clearly shown in Figure 8. The results for medium datasets are shown in Table 7, where our proposed DDPG algorithm is compared with the baseline algorithms LSTM, DQN and A2C. For a dataset of 400 to 650 tasks, proposed work has reduced the energy consumption of DDPG when compared with the existing algorithms. The comparison of baseline algorithms and the proposed work is clearly shown in Figure 9. The results for large datasets are shown in Table 8, where our proposed DDPG algorithm is compared with the baseline algorithms LSTM, DQN and A2C. For a dataset of 700 to 1000 tasks, the proposed DDPG reduces the energy consumption compared to the existing algorithms. The comparison of baseline algorithms and the proposed work is clearly shown in Figure 10.

D. FAULT TOLERANCE

Fault tolerance was a key metric in fog computing, especially in healthcare systems where reliability and continuity are crucial. It measures the system's ability to handle task failures

TABLE 6. Energy consumption for small datasets.

Datasets	LSTM	DQN	A2C	DDPG
100	41.9	32.9	28.9	24.2
150	52.5	46.9	43.5	37.3
200	61.8	57.8	54.7	45.8
250	71.5	68.9	64.8	54.8
300	80.6	77.4	73.5	59.9
350	93.5	89.5	83.3	67.1

TABLE 7. Energy consumption for medium datasets.

Datasets	LSTM	DQN	A2C	DDPG
400	109.5	97.8	86.9	78.7
450	118.8	109.5	91.6	82.9
500	126.5	124.8	104.5	95.2
550	148.8	138.5	119.2	108.9
600	161.9	152.8	126.6	114.2
650	178.6	163.4	136.8	124.4

TABLE 8. Energy consumption for large datasets.

Datasets	LSTM	DQN	A2C	DDPG
700	184.5	168.8	154.7	133.7
750	198.8	187.4	162.2	142.8
800	229.5	210.8	171.6	150.2
850	242.4	223.5	186.2	167.4
900	268.5	237.8	192.4	172.8
950	283.8	248.4	208.6	189.2
1000	302.8	269.7	217.8	198.5

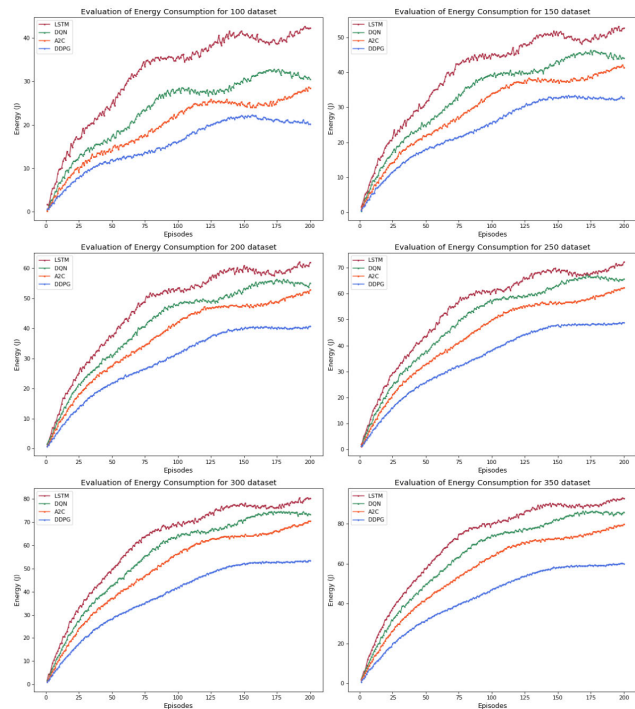


FIGURE 8. Energy consumption of 100_350 datasets.

or interruptions without compromising overall performance. High fault tolerance ensures that crucial tasks are dynamically rerouted, reducing downtime and preventing data loss or service disruptions. By using redundancy mechanisms such as

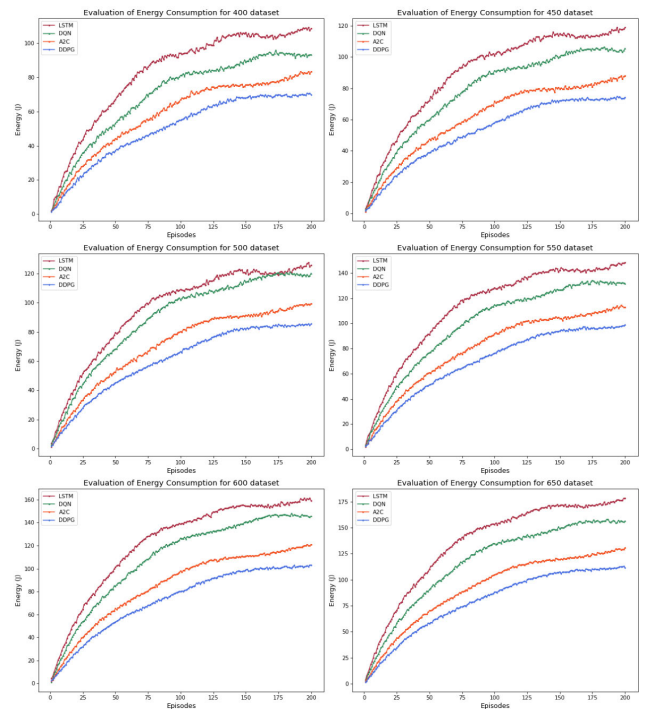


FIGURE 9. Energy consumption of 400_650 datasets.

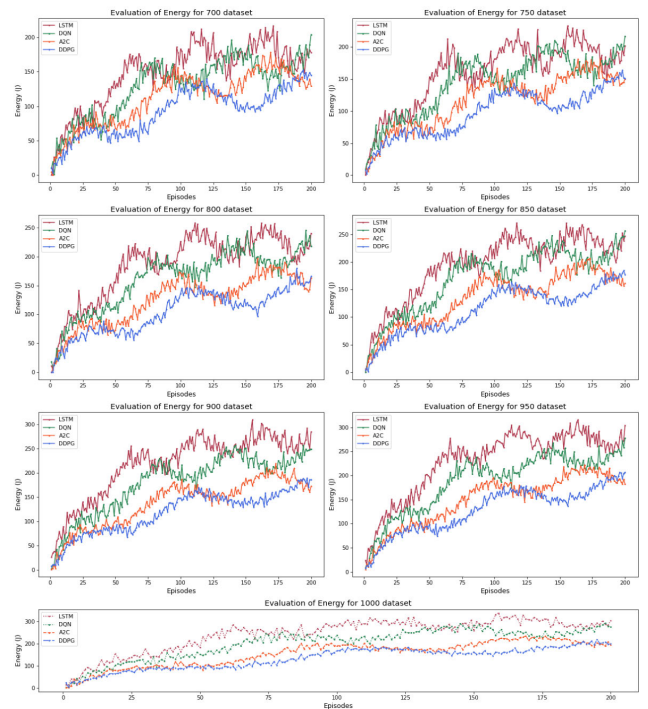


FIGURE 10. Energy consumption of 700_1000 datasets.

advanced scheduling algorithms and reinforcement learning-based approaches, fog computing systems can effectively handle node failures and resource availability. Fault tolerance was assessed using experimental configurations and the dataset was divided into three groups: small (dataset 100 to

350), medium (dataset 400 to 650), and large (dataset 700 to 1000). The results for small datasets are shown in Table 9, where our proposed DDPG algorithm is compared with the baseline algorithms LSTM, DQN, and A2C. For datasets of 100 to 350 tasks, the proposed work DDPG has reduced fault tolerance when compared with the exiting algorithms. The comparison of baseline algorithms and the proposed work is clearly shown in Figure 11. The results for medium datasets are shown in Table 10, where our proposed DDPG algorithm is compared with the baseline algorithms LSTM, DQN, and A2C. For datasets of 400 to 650 tasks, the proposed work DDPG has reduced fault tolerance when compared with the baseline algorithms. The comparison of existing algorithms and the proposed work clearly shows the fault tolerance in Figure 12. The results for large datasets are shown in Table 11. Our proposed DDPG algorithm is compared with the LSTM, DQN and A2C baseline algorithms. For a dataset of 700 to 1000 tasks, the proposed DDPG has reduced the fault tolerance when compared to the baseline algorithms. The comparison of baseline algorithms and the proposed work clearly shows the fault tolerance in Figure 13.

TABLE 9. Fault tolerance for small datasets.

Datasets	LSTM	DQN	A2C	DDPG
100	83.6	79.8	78.8	64.2
150	81.4	77.4	72.5	62.5
200	75.9	72.1	68.6	57.8
250	67.1	65.5	62.4	51.9
300	64.6	62.6	59.6	49.7
350	61.4	59.5	64.9	47.2

TABLE 10. Fault tolerance medium datasets.

Datasets	LSTM	DQN	A2C	DDPG
400	59.8	56.9	51.4	45.6
450	56.6	54.4	50.9	43.8
500	53.2	51.3	47.6	41.2
550	51.6	49.6	45.8	39.5
600	47.9	45.9	42.5	36.5
650	41.6	39.6	36.7	31.8

TABLE 11. Fault tolerance for large datasets.

Datasets	LSTM	DQN	A2C	DDPG
700	42.9	39.9	35.6	29.5
750	45.6	33.6	39.8	27.7
800	39.5	30.5	36.4	24.2
850	48.3	38.6	28.4	21.9
900	49.6	37.5	27.6	19.9
950	45.4	32.6	24.5	18.4
1000	38.7	28.5	22.4	15.7

E. TRUST

Trust is a fundamental metric in fog computing, especially in healthcare systems where reliability and data integrity are crucial. It assesses the level of confidence in the system's ability to execute tasks accurately and securely, ensuring that

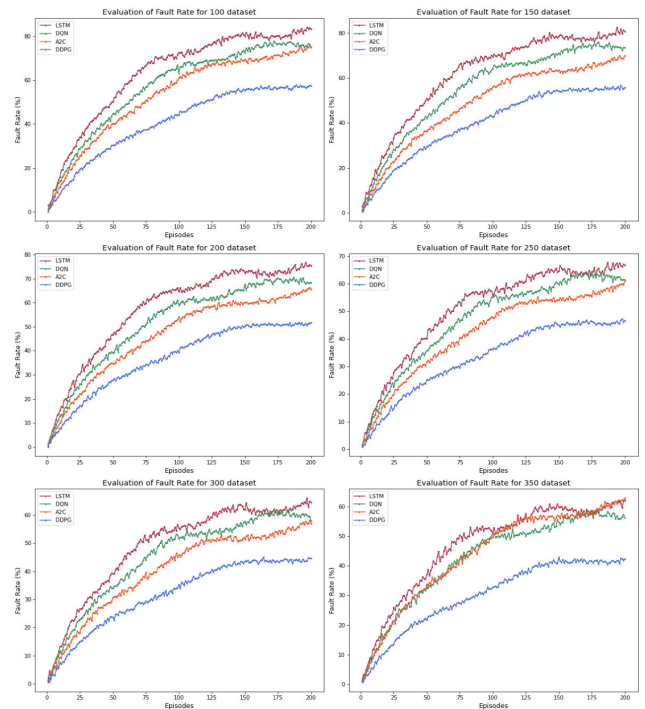


FIGURE 11. Fault tolerance of 100_350 datasets.

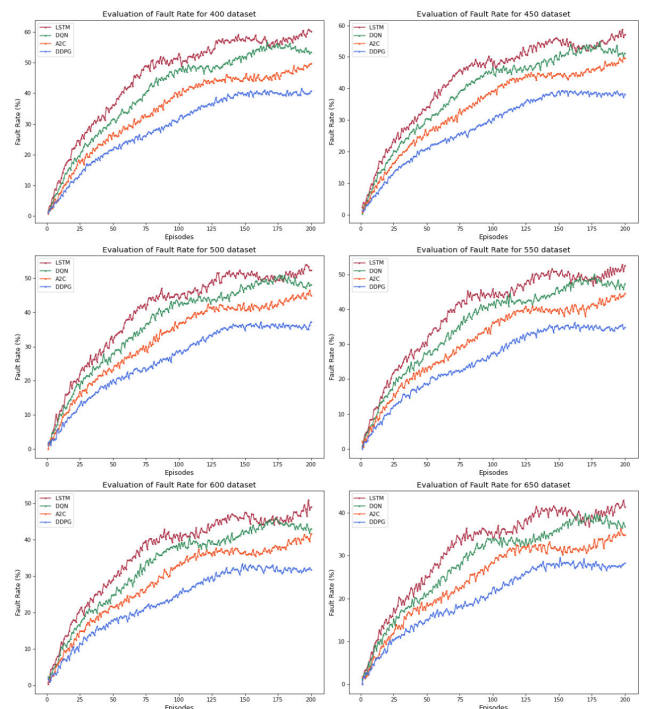


FIGURE 12. Fault tolerance of 400_650 datasets.

they meet predefined performance standards. Higher trust levels increase the reliability of task scheduling and resource allocation policies, encouraging trusted interactions between fog nodes. Trust analyzed the experimental configurations and divided the dataset into three groups: small (dataset

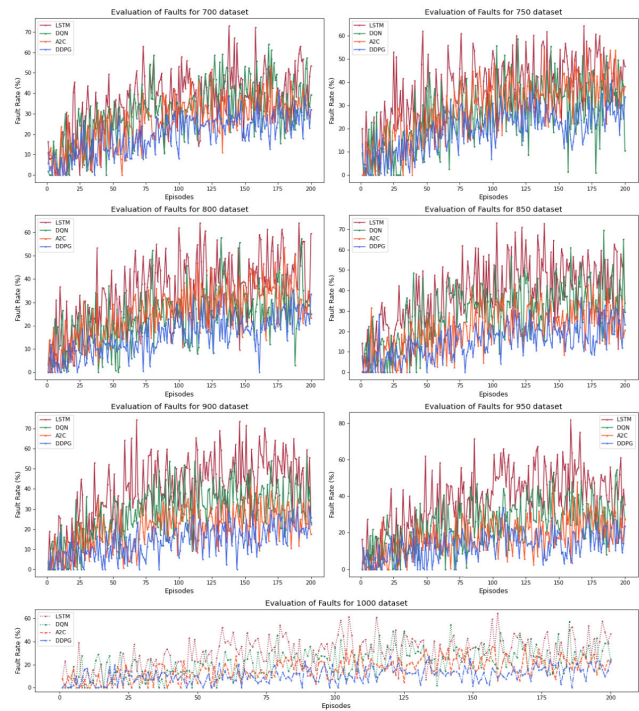


FIGURE 13. Fault tolerance of 700_1000 datasets.

TABLE 12. Trust for small datasets.

Datasets	LSTM	DQN	A2C	DDPG
100	0.645	0.798	0.864	0.935
150	0.664	0.768	0.854	0.951
200	0.669	0.762	0.864	0.926
250	0.668	0.786	0.898	0.912
300	0.668	0.745	0.859	0.962
350	0.666	0.774	0.865	0.975

TABLE 13. Trust for medium datasets.

Datasets	LSTM	DQN	A2C	DDPG
400	0.689	0.798	0.884	0.923
450	0.684	0.784	0.855	0.916
500	0.665	0.756	0.847	0.963
550	0.688	0.758	0.854	0.924
600	0.698	0.768	0.888	0.931
650	0.689	0.795	0.846	0.962

TABLE 14. Trust for large datasets.

Datasets	LSTM	DQN	A2C	DDPG
700	0.684	0.778	0.884	0.943
750	0.689	0.787	0.895	0.986
800	0.688	0.781	0.896	0.972
850	0.684	0.747	0.898	0.964
900	0.654	0.767	0.859	0.983
950	0.667	0.784	0.885	0.963
1000	0.684	0.786	0.887	0.978

100 to 350), medium (dataset 400 to 650), and large (dataset 700 to 1000). The results for the small datasets are shown in Table12. Our proposed DDPG algorithms are compared against the baseline algorithms LSTM, DQN, and A2C. The

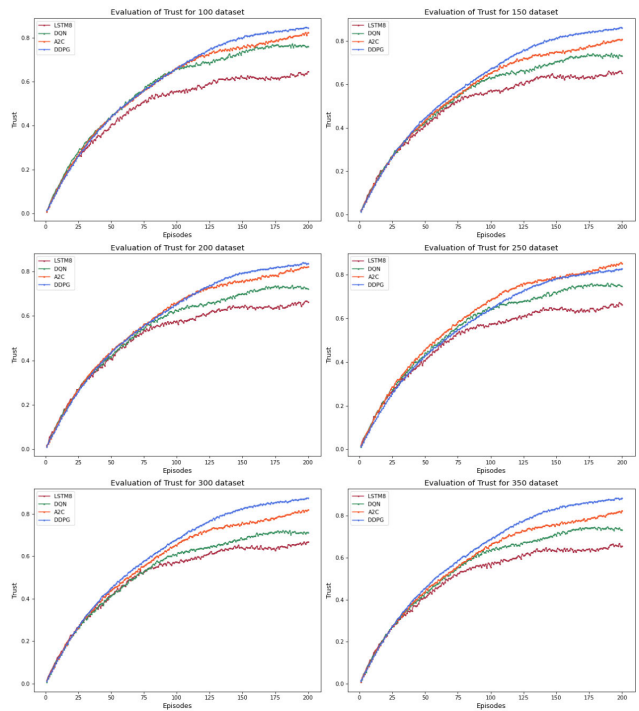


FIGURE 14. Trust of 100_350 datasets.

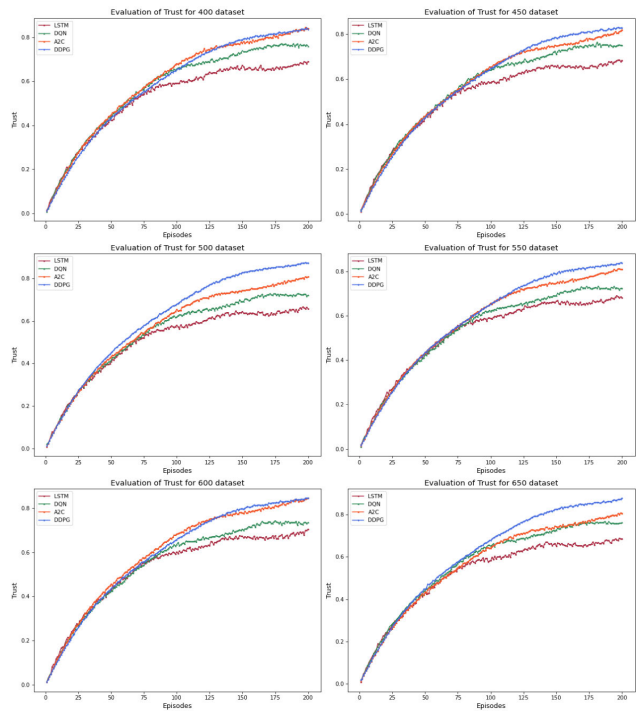


FIGURE 15. Trust of 400_650 datasets.

proposed task DDPG increases the confidence compared to the existing algorithms for a dataset of tasks from 100 to 350. The comparison of the existing algorithms and the proposed tasks is clearly shown in Figure 14. The results for medium datasets are shown in Table13, where our proposed DDPG

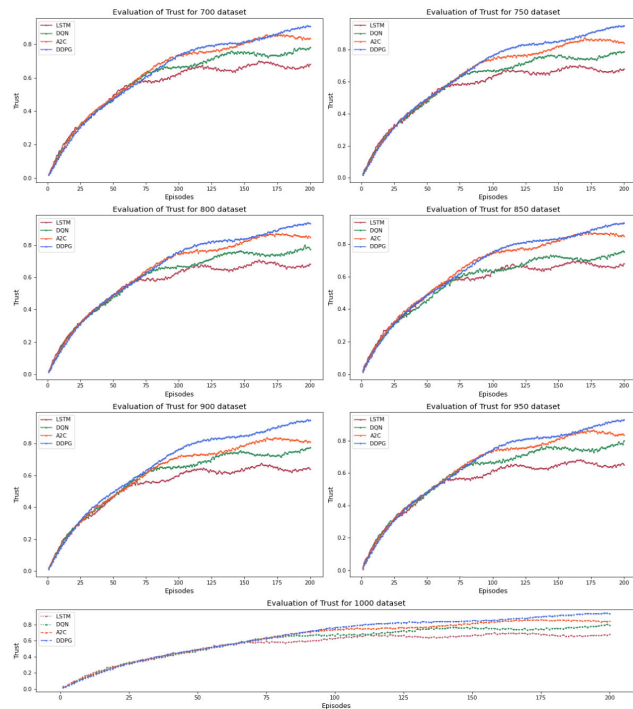


FIGURE 16. Trust of 700_1000 datasets.

algorithms are compared with the LSTM, DQN, and A2C baseline algorithms. The proposed task DDPG increases the confidence compared to the existing algorithms for a dataset of tasks from 400 to 650. The comparison of the baseline algorithms and the proposed tasks is clearly shown in Figure 15. The results for large datasets are shown in Table 14, where our proposed DDPG algorithms are compared with the baseline algorithms LSTM, DQN, and A2C. For a dataset of 700 to 1000 tasks, the proposed DDPG increases the trust compared to the existing algorithms. The comparison of the existing algorithms and the proposed work is clearly shown in Figure 16.

VII. CONCLUSION

This work represents the new framework behavior deep reinforcement learning for task scheduling and load balancing optimization in fog computing, especially in crucial healthcare applications. The proposed work overcomes the inherent shortcomings of traditional cloud computing infrastructures that suffer from high latency and reduced real-time processing, a crucial aspect in the healthcare sector where data processing speed is directly proportional to patient care outcomes. We propose a model that uses strategic placement of processing nodes close to data generation sources without transmission delays and fast response times. Our results showed significant performance improvements through control simulations in the SimPy environment. Specifically, it reduced task completion time by 30%, reduced operational latency by 40%, and improved fault tolerance by 25% compared to traditional cloud-based task scheduling

techniques. Moreover, the versatility of the DRL model has been extensively tested in various applications beyond healthcare, including industrial automation and smart traffic management. These tests further confirmed the model's ability to improve data throughput and operational response times in diverse real-time data processing contexts. This demonstrates the remarkable benefit of integrating DRL of task scheduling development of fog computing architectures. The improved performance will support mission-critical healthcare operations in addition to providing improved efficiency for general IoT networks in other industries. For future work, we propose to further integrate the hybrid cloud-fog model to analyze whether it can provide a well-balanced solution for dealing with large datasets while maintaining the low-latency characteristics of fog computing. Further studies on the synergies between DRL and other emerging strategies for machine learning will enhance further improvements in scheduling efficiency and resource management.

REFERENCES

- [1] A. Arunarani, D. Manjula, and V. Sugumaran, "Task scheduling techniques in cloud computing: A literature survey," *Future Gener. Comput. Syst.*, vol. 91, pp. 407–415, Feb. 2019.
- [2] P. Singh, M. Dutta, and N. Aggarwal, "A review of task scheduling based on meta-heuristics approach in cloud computing," *Knowl. Inf. Syst.*, vol. 52, no. 1, pp. 1–51, Jul. 2017.
- [3] M. Hosseinzadeh, E. Azhir, J. Lansky, S. Mildeeva, O. H. Ahmed, M. H. Malik, and F. Khan, "Task scheduling mechanisms for fog computing: A systematic survey," *IEEE Access*, vol. 11, pp. 50994–51017, 2023.
- [4] M. Abdel-Basset, N. Moustafa, R. Mohamed, O. M. Elkomy, and M. Abouhawwash, "Multi-objective task scheduling approach for fog computing," *IEEE Access*, vol. 9, pp. 126988–127009, 2021.
- [5] S. D. Vispute and P. Vashist, "Energy-efficient task scheduling in fog computing based on particle swarm optimization," *Social Netw. Comput. Sci.*, vol. 4, no. 4, p. 391, May 2023.
- [6] R. Ghafari and N. Mansouri, "An efficient task scheduling in fog computing using improved artificial hummingbird algorithm," *J. Comput. Sci.*, vol. 74, Dec. 2023, Art. no. 102152.
- [7] F. A. Saif, R. Latip, Z. M. Hanapi, and K. Shafinah, "Multi-objective grey wolf optimizer algorithm for task scheduling in cloud-fog computing," *IEEE Access*, vol. 11, pp. 20635–20646, 2023.
- [8] S. M. Hosseini, M. H. Shirvani, and H. Motameni, "Multi-objective discrete cuckoo search algorithm for optimization of bag-of-tasks scheduling in fog computing environment," *Comput. Electr. Eng.*, vol. 119, Oct. 2024, Art. no. 109480.
- [9] A. A. Mutlag, M. K. A. Ghani, O. Mohd, K. H. Abdulkareem, M. A. Mohammed, M. Alharbi, and Z. J. Al-Araji, "A new fog computing resource management (FRM) model based on hybrid load balancing and scheduling for critical healthcare applications," *Phys. Commun.*, vol. 59, Aug. 2023, Art. no. 102109.
- [10] E. M. Elshahed, R. M. Abdelmoneem, E. Shaaban, H. A. Elzahed, and S. M. Al-Tabbakh, "Prioritized scheduling technique for healthcare tasks in cloud computing," *J. Supercomput.*, vol. 79, no. 5, pp. 4895–4916, Mar. 2023.
- [11] A. Jayanetti, S. Halgamuge, and R. Buyya, "Deep reinforcement learning for energy and time optimized scheduling of precedence-constrained tasks in edge-cloud computing environments," *Future Gener. Comput. Syst.*, vol. 137, pp. 14–30, Dec. 2022.
- [12] A. Lakhan, J. Nedoma, M. A. Mohammed, M. Deveci, M. Fajkus, H. A. Marhoon, S. Memon, and R. Martinek, "Fiber-optics IoT healthcare system based on deep reinforcement learning combinatorial constraint scheduling for hybrid telemedicine applications," *Comput. Biol. Med.*, vol. 178, Aug. 2024, Art. no. 108694.
- [13] M. M. Kamruzzaman, S. Alanazi, M. Alruwaili, I. Alrashdi, Y. Alhwaiti, and N. Alshammari, "Fuzzy-assisted machine learning framework for the fog-computing system in remote healthcare monitoring," *Measurement*, vol. 195, May 2022, Art. no. 111085.

- [14] A. Jangra and N. Mangla, "An efficient load balancing framework for deploying resource scheduling in cloud based communication in healthcare," *Meas., Sensors*, vol. 25, Feb. 2023, Art. no. 100584.
- [15] S. Khan, I. A. Shah, M. F. Nadeem, S. Jan, T. Whangbo, and S. Ahmad, "Optimal resource allocation and task scheduling in fog computing for Internet of Medical Things applications," *HCIS*, vol. 13, p. 56, Jan. 2023.
- [16] M. A. Ibrahim and S. Askar, "An intelligent scheduling strategy in fog computing system based on multi-objective deep reinforcement learning algorithm," *IEEE Access*, vol. 11, pp. 133607–133622, 2023.
- [17] Z. Gao, L. Yang, and Y. Dai, "Fast adaptive task offloading and resource allocation via multiagent reinforcement learning in heterogeneous vehicular fog computing," *IEEE Internet Things J.*, vol. 10, no. 8, pp. 6818–6835, Apr. 2023.
- [18] P. Verma, R. Tiwari, W.-C. Hong, S. Upadhyay, and Y.-H. Yeh, "FETCH: A deep learning-based fog computing and IoT integrated environment for healthcare monitoring and diagnosis," *IEEE Access*, vol. 10, pp. 12548–12563, 2022.
- [19] D. H. Abdulazeez and S. K. Askar, "Offloading mechanisms based on reinforcement learning and deep learning algorithms in the fog computing environment," *IEEE Access*, vol. 11, pp. 12555–12586, 2023.
- [20] S. Tosun, "Energy- and reliability-aware task scheduling onto heterogeneous MPSoC architectures," *J. Supercomput.*, vol. 62, no. 1, pp. 265–289, Oct. 2012.
- [21] R. Siyadatzaheh, F. Mehrafrooz, M. Ansari, B. Safaei, M. Shafique, J. Henkel, and A. Ejlali, "ReLIEF: A reinforcement learning-based real-time task assignment strategy in emerging fault-tolerant fog computing," *IEEE Internet Things J.*, pp. 10752–10763, 2023.
- [22] P. Johri, V. Balu, B. Jayaprakash, A. Jain, C. Thacker, and A. Kumari, "Quality of service-based machine learning in fog computing networks for e-healthcare services with data storage system," *Soft Comput.*, pp. 1–13, Aug. 2023.
- [23] F. M. Talaat, H. A. Ali, M. S. Saraya, and A. I. Saleh, "Effective scheduling algorithm for load balancing in fog environment using CNN and MPSO," *Knowl. Inf. Syst.*, vol. 64, no. 3, pp. 773–797, Mar. 2022.
- [24] V. Jain and B. Kumar, "QoS-aware task offloading in fog environment using multi-agent deep reinforcement learning," *J. Netw. Syst. Manage.*, vol. 31, no. 1, p. 7, Jan. 2023.
- [25] F. M. Talaat, "Effective prediction and resource allocation method (EPRAM) in fog computing environment for smart healthcare system," *Multimedia Tools Appl.*, vol. 81, no. 6, pp. 8235–8258, Mar. 2022.
- [26] G. Goel and R. Tiwari, "IGWOA: Improved grey wolf optimization algorithm for resource scheduling in cloud-fog environment for delay-sensitive applications," *Peer Peer Netw. Appl.*, vol. 17, no. 3, pp. 1768–1790, May 2024.
- [27] N. Arivazhagan, K. Somasundaram, D. V. Babu, M. G. Nayagam, R. M. Bommi, G. B. Mohammad, P. R. Kumar, Y. Natarajan, V. J. Arulkarthick, V. K. Shanmuganathan, K. Srihari, M. R. Vignesh, and V. P. Sundramurthy, "Cloud-Internet of Health Things (IOHT) task scheduling using hybrid moth flame optimization with deep neural network algorithm for e healthcare systems," *Sci. Program.*, vol. 2022, pp. 1–12, Jan. 2022.
- [28] K. Alatoun, K. Matrouk, M. A. Mohammed, J. Nedoma, R. Martinek, and P. Zmij, "A novel low-latency and energy-efficient task scheduling framework for Internet of Medical Things in an edge fog cloud system," *Sensors*, vol. 22, no. 14, p. 5327, Jul. 2022.
- [29] J. Lim, "Latency-aware task scheduling for IoT applications based on artificial intelligence with partitioning in small-scale fog computing environments," *Sensors*, vol. 22, no. 19, p. 7326, Sep. 2022.
- [30] S. S. Tripathy, M. Rath, N. Tripathy, D. S. Roy, J. S. A. Francis, and S. Bebortta, "An intelligent health care system in fog platform with optimized performance," *Sustainability*, vol. 15, no. 3, p. 1862, Jan. 2023.
- [31] C. L. Blake. (1998). *UCI Repository of Machine Learning Databases*. [Online]. Available: <http://www.ics.uci.edu/~mllearn/MLRepository.html>



PRASHANTH CHOPPARA received the B.Tech. degree from Acharya Nagarjuna University, in 2018, and the M.Tech. degree from JNTUK University, in 2021. Currently, he is pursuing the Ph.D. degree with the School of Computer Science and Engineering (SCOPE), VIT-AP University, Andhra Pradesh, India. His research interests include cloud computing, fog computing, machine learning, deep learning, and reinforcement learning.



BOMMAREDDY LOKESH (Member, IEEE) received the Bachelor of Technology degree in information technology from the PVP Siddhartha Institute of Technology, Vijayawada, in 2011, the Master of Technology degree in computer science and technology from Andhra University College of Engineering, in 2013, and the Ph.D. degree in computer science and engineering from the National Institute of Technology Puducherry, Karaikal, in 2022. He is currently an Assistant Professor with the School of Computer Science and Engineering, VIT-AP University, Amaravati, India. His research interests include software-defined networking, machine learning, the Internet of Things, network function virtualization, and 5G communications.

...